CSA47 – Deep Learning Assignment 1

Name : Murali Manokar D K

Reg. No.: 192324044

Dataset: PlantVillage – *Tomato* subset (10 classes: 9 diseases + healthy, ≈18,160 images), resized to 64 × 64 grayscale → 4,096 features. Task 12 additionally uses all 38 PlantVillage classes.

Conventions used throughout

- Every formula is stated before it is used; each numeric result is printed to 4 decimal places.
- Information-theoretic quantities use log base 2 (bits).
- Hyper-parameters are chosen on the validation set; the test set is touched only in Task 9.

```
import os, re, time, pickle, subprocess, warnings
import numpy as np, pandas as pd, matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score,
StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.pipeline import make_pipeline
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics.pairwise import euclidean_distances
from sklearn.metrics import (accuracy_score,
precision_recall_fscore_support,
                             confusion_matrix, ConfusionMatrixDisplay)
warnings.filterwarnings('ignore')
np.set_printoptions(precision=4, suppress=True)
pd.set_option('display.float_format', lambda v: f'{v:.4f}')
plt.rcParams.update({'figure.dpi': 110, 'axes.grid': True,
'grid.alpha': .3})

SEED = 42
rng = np.random.default_rng(SEED)
IMG = 64
DATA_DIR = None          # optional: your own PlantVillage folder (one
sub-folder per class); None = download from GitHub
BASE = '/content' if os.path.isdir('/content') else '.'
CACHE = f'{BASE}/agrivision_cache.npz'
```

```
PV_ROOT = f'{BASE}/PlantVillage-Dataset'
TEST_MODE = os.environ.get('AGRI_TEST') == '1'    # synthetic smoke-test only; leave unset in Colab
PER_CLASS_38 = 150        # images per class for the 38-class experiment (Task 12)
def f4(x): return float(np.round(x, 4))
```

Data Collection & Preprocessing:

Downloads PlantVillage from GitHub (sparse checkout: only the files needed) or reads a local folder, converts every image to grayscale, resizes to 64 × 64 and scales pixels to $[0, 1)$:

$$x_{ij} = \frac{\text{pixel}_{ij}}{255} \in [0, 1),\ x = \text{vec}(X) \in R^{4096}$$

Keeps the 10 tomato classes (Tasks 1–11) and a per-class sample of all 38 classes (Task 12). Arrays are cached so re-runs are instant.

```
def fetch_plantvillage(root, per_other):
    # sparse, blob-less clone: downloads only the chosen image files (all tomato + per_other per other class)
    print('Downloading PlantVillage images from GitHub (one-time, a few minutes)...')
    url = 'https://github.com/spMohanty/PlantVillage-Dataset.git'
    if not os.path.isdir(root):
        subprocess.run(['git', 'clone', '-q', '--depth', '1', '--filter=blob:none', '--no-checkout', url, root], check=True)
    files = subprocess.run(['git', '-C', root, 'ls-tree', '-r', '--name-only', 'HEAD', 'raw/grayscale'],
                           capture_output=True, text=True, check=True).stdout.splitlines()
    by_cls = {}
    for f in files: by_cls.setdefault(f.split('/')[2], []).append(f)
    pick = [f for c, fs in sorted(by_cls.items())
            for f in (fs if c.lower().startswith('tomato') else sorted(fs)[:per_other])]
    pats = '\n'.join('/' + re.sub(r'([\[\]*?\\!#])', r'\\\1', f) for f in pick)
    subprocess.run(['git', '-C', root, 'sparse-checkout', 'set', '--no-cone', '--stdin'], input=pats, text=True, check=True)
    subprocess.run(['git', '-C', root, 'checkout', '-q', 'HEAD'], check=True)
    return os.path.join(root, 'raw', 'grayscale')

def load_folder(root):
    from PIL import Image
    names = sorted(d for d in os.listdir(root) if os.path.isdir(os.path.join(root, d)))
    Xs, ys = [], []
```

```
    for i, n in enumerate(names):
        for f in sorted(os.listdir(os.path.join(root, n))):
            try:
                im = Image.open(os.path.join(root, n,
f)).convert('L').resize((IMG, IMG))
                Xs.append(np.asarray(im, np.float32) / 255.0);
ys.append(i)
            except Exception:
                pass
    return np.stack(Xs), np.array(ys), names

def make_synthetic(K, per):
    # smoke-test stand-in only (class = position of a dark blob on a
bright leaf)
    yy, xx = np.mgrid[0:IMG, 0:IMG]
    Xs, ys = [], []
    for c in range(K):
        cy, cx = 12 + (c * 7) % 40, 12 + (c * 13) % 40
        for _ in range(per):
            img = 0.6 + 0.1 * rng.standard_normal() - 0.4 * np.exp(-
((yy - cy - rng.normal(0, 3))**2 + (xx - cx - rng.normal(0, 3))**2) /
60)
            Xs.append(np.clip(img + 0.05 * rng.standard_normal((IMG,
IMG)), 0, 1).astype(np.float32)); ys.append(c)
    return np.stack(Xs), np.array(ys), [f'Tomato___synthetic_{c}' for
c in range(K)]

if TEST_MODE:
    Xall, yall, names_all = make_synthetic(12, 260)
elif os.path.exists(CACHE):
    d = np.load(CACHE, allow_pickle=True); Xall, yall, names_all =
d['X'], d['y'], list(d['names'])
else:
    Xall, yall, names_all = load_folder(DATA_DIR or
fetch_plantvillage(PV_ROOT, PER_CLASS_38))
    # keep tomato images + a per-class sample of all 38 classes (keeps
the cache small)
    keep = np.zeros(len(yall), bool)
    for c, n in enumerate(names_all):
        idx = np.where(yall == c)[0]
        keep[idx if n.lower().startswith('tomato') else
rng.permutation(idx)[:PER_CLASS_38]] = True
    Xall, yall = Xall[keep], yall[keep]
    np.savez_compressed(CACHE, X=Xall, y=yall,
names=np.array(names_all))

# ---- 38-class sample (Task 12)
idx38 = np.concatenate([rng.permutation(np.where(yall == c)[0])
[:PER_CLASS_38] for c in np.unique(yall)])
X38, y38 = Xall[idx38].reshape(len(idx38), -1), yall[idx38]
```

```
# ---- Tomato subset (Tasks 1-11)
tom = [c for c, n in enumerate(names_all) if
n.lower().startswith('tomato')]
if TEST_MODE: tom = tom[:10]
mask = np.isin(yall, tom)
X_img = Xall[mask]                                    # (N, 64, 64)
remap = {c: i for i, c in enumerate(tom)}
y = np.array([remap[c] for c in yall[mask]])          # (N,)
CLASSES = [names_all[c].split('___')[-1] for c in tom]
K = len(CLASSES)
X = X_img.reshape(len(X_img), -1)                     # (N, 4096)
print('Tomato subset:', X_img.shape, '->', X.shape, '| classes:', K)
print(pd.Series(np.bincount(y), index=CLASSES, name='images'))
print('38-class sample:', X38.shape, 'classes:', len(np.unique(y38)))
```

```
Downloading PlantVillage images from GitHub (one-time, a few
minutes)...
Tomato subset: (18160, 64, 64) -> (18160, 4096) | classes: 10
Bacterial_spot                          2127
Early_blight                            1000
Late_blight                             1909
Leaf_Mold                                952
Septoria_leaf_spot                      1771
Spider_mites Two-spotted_spider_mite    1676
Target_Spot                             1404
Tomato_Yellow_Leaf_Curl_Virus           5357
Tomato_mosaic_virus                      373
healthy                                 1591
Name: images, dtype: int64
38-class sample: (5700, 4096) classes: 38
```

Train / Validation / Test Split

Stratified split keeps every class's proportion equal in all three subsets:

$$P_{train}(y=k) \approx P_{val}(y=k) \approx P_{test}(y=k) \forall\ k$$

70 % train (fit), 15 % validation (choose hyper-parameters), 15 % test (locked until Task 9).

```
# Stratified split 70 / 15 / 15 (test set is locked away until Task 9)
X_tv, X_te, y_tv, y_te = train_test_split(X, y, test_size=0.15,
stratify=y, random_state=SEED)
X_tr, X_va, y_tr, y_va = train_test_split(X_tv, y_tv,
test_size=0.15/0.85, stratify=y_tv, random_state=SEED)
print('train', X_tr.shape, '| val', X_va.shape, '| test', X_te.shape)
split_tab = pd.DataFrame({s: np.bincount(v, minlength=K) / len(v) for
s, v in
                          [('train', y_tr), ('val', y_va), ('test',
```

```
y_te)]}, index=CLASSES)
split_tab
```

```
train (12712, 4096) | val (2724, 4096) | test (2724, 4096)
```


```
{"summary":"{\n  \"name\": \"split_tab\",\n  \"rows\": 10,\n
\"fields\": [\n    {\n      \"column\": \"train\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.0742957343579965,\n        \"min\": 0.02053178099433606,\n
\"max\": 0.2949968533668974,\n        \"num_unique_values\": 10,\n
\"samples\": [\n          0.02053178099433606,\n
0.05506607929515418,\n          0.09235368156073002\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"val\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.07433539487736773,\n
\"min\": 0.020558002936857563,\n        \"max\": 0.29515418502202645,\
n        \"num_unique_values\": 10,\n        \"samples\": [\n
0.020558002936857563,\n          0.05506607929515418,\n
0.092143906020558\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"test\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.07421584429550918,\n        \"min\": 0.020558002936857563,\
n        \"max\": 0.2947870778267254,\n\"num_unique_values\":
10,\n        \"samples\": [\n          0.020558002936857563,\n
0.05506607929515418,\n          0.092143906020558\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n ]\n}","type":"dataframe","variable_name":"split_tab"}
```


Task 1 — Problem Structure and Data Representation (4 marks)

Learning-task decomposition (Mitchell's T, P, E)

Component	AgriVision	How it constrains the solution
Task T	Multi-class classification $f: R^{4096} \rightarrow \{1, \ldots, K\}$	Needs a probabilistic output over K classes → softmax / cross-entropy (Task 3, 8)
Input x	64×64 grayscale leaf photo, $x \in [0, 1)^{4096}$	High-dimensional, spatially correlated → redundancy (Task 2), curse of dimensionality (Task 11)
Output y	Class label $y \in \{0..K-1\}$, one-hot $e_y \in \{0, 1\}^K$	Imbalanced classes → accuracy alone misleading
Performance P	Accuracy + macro precision/recall/F1 + confusion matrix	Macro-F1 weights rare diseases equally; recall matters (a missed disease spreads)
Experience E	Labelled PlantVillage images	Lab images, uniform background → optimistic vs field photos

Tensor organisation

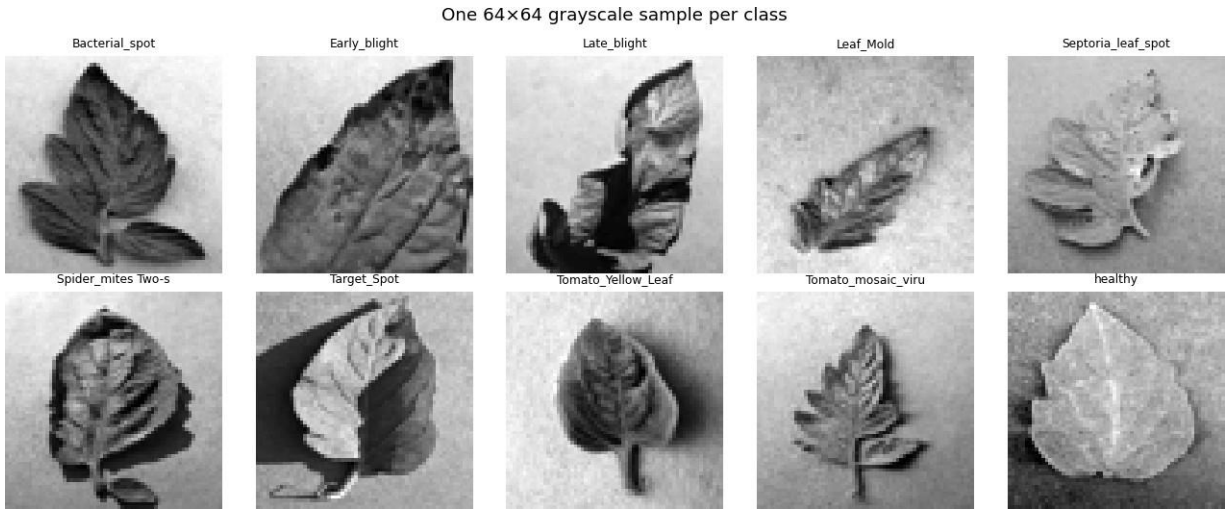
Object	Type	Shape	Why the shape matters
one label y_i	scalar	()	integer class index; one-hot vector (K ,) for cross-entropy
one image	matrix → vector	(64,64) → (4096,)	flattening discards 2-D neighbourhood
mini-batch	matrix / 4-D tensor	(32,4096) or (32,64,64,1)	rows = samples, enables XW in one matmul
dataset	design matrix	(N,4096)	$X^\top X$ is 4096×4096 * PCA cost
labels	vector / matrix	(N,) or (N,K)	$P-Y$ appears in the gradient (Task 8)

Norms (for $x \in R^n$): $\|x\|_1 = \sum_i |x_i|$, $\|x\|_2 = \sqrt{\sum_i x_i^2}$, $\|x\|_\infty = \max_i |x_i|$

```
fig, ax = plt.subplots(2, 5, figsize=(12, 5))
for c, a in enumerate(ax.flat[:K]):
    a.imshow(X_img[np.where(y == c)[0][0]], cmap='gray');
a.set_title(CLASSES[c][:18], fontsize=8); a.axis('off')
plt.suptitle('One 64×64 grayscale sample per class');
plt.tight_layout(); plt.show()

x_one = X_tr[0]
Y_onehot = np.eye(K)[y_tr]
shapes = pd.DataFrame([
    ('scalar label y_0', np.shape(y_tr[0]), 'scalar'),
    ('one image (matrix)', X_img[0].shape, 'matrix'),
    ('one image (vector)', x_one.shape, 'vector'),
    ('mini-batch (flat)', X_tr[:32].shape, 'matrix'),
    ('mini-batch (CNN)', X_tr[:32].reshape(32, IMG, IMG, 1).shape, '4-
D tensor'),
    ('dataset design matrix', X_tr.shape, 'matrix'),
    ('labels', y_tr.shape, 'vector'),
    ('labels one-hot', Y_onehot.shape, 'matrix')], columns=['object',
```

```
'shape', 'type'])
display(shapes)
```




```
{"summary":"{\n \"name\": \"shapes\",\n \"rows\": 8,\n \"fields\": [\n{\n\"column\":
\"object\",\n\"properties\": {\n
\"dtype\": \"string\",\n                    \"num_unique_values\": 8,\n
\"samples\": [\n design matrix\",\n     \"one image (matrix)\",\n                    \"dataset
                    \"scalar label v  0\"\n                    ],\n
\"semantic_type\": \"\",\n\"description\": \"\"\n}\
n},\n{\n\"column\": \"shape\",\n\"properties\":    {\ n\"dtype\": \"string\",\n\"num_unique_values\":
8.\n
\"samples\": [\n                    [\n                    64,\n                    64\n
],\n                    [\n                    12712,\n                    4096\n                    ],\n
[]\n],\n\"semantic_type\": \"\",\n
\"description\": \"\"\n}\n},\n{\n\"column\":
\"type\",\n\"properties\": {\n\"dtype\": \"string\",\n
\"num_unique_values\": 4,\n\"samples\": [\n
\"matrix\",\n\"4-D tensor\",\n\"scalar\"\ n],\n\"semantic_type\": \"\",\n
\"description\": \"\"\n}\n}\n ]\ n}","type":"dataframe","variable_name":"shapes"}
```


Norms of One Feature Vector

L^1 norm

$$\| x \|_1 = \sum_{i=1}^{n} | x_i)$$

L^2 norm

$$\| x \|_2 = \sqrt{\sum_{i=1}^{n} x^2{}_i}$$

Max ($L\infty$) norm

$$\| x \|_\infty = \max_i | x_i)$$

Computed for the same image in raw $[0, 1)$ scale, after standardisation, and for the difference of

two images; cross-checked with `numpy.linalg.norm`.

```
# Norms of the same feature vector, raw [0,1] scale vs standardised
scaler_px = StandardScaler().fit(X_tr)
x_std = scaler_px.transform(x_one[None])[0]
def norms(v): return [f4(np.abs(v).sum()), f4(np.sqrt((v**2).sum())),
f4(np.abs(v).max())]
nt = pd.DataFrame([norms(x_one), norms(x_std), norms(X_tr[0] -
X_tr[1])],
                  index=['x (pixels in [0,1])', 'x standardised', 'x_0
- x_1 (difference)'],
                  columns=['L1', 'L2', 'Linf'])
display(nt)
print('Check (numpy.linalg.norm):', [f4(np.linalg.norm(x_one, o)) for
o in (1, 2, np.inf)])
```


```
{"summary":"{\n  \"name\": \"nt\",\n  \"rows\": 3,\n  \"fields\": [\n
{\n      \"column\": \"L1\",\n      \"properties\": {\n
\"dtype\": \"number\",\n         \"std\": 1120.7547542078637,\n
\"min\": 489.6077880859375,\n         \"max\": 2654.493408203125,\n
\"num_unique_values\": 3,\n         \"samples\": [\n
2075.2666015625,\n           2654.493408203125,\n
489.6077880859375\n         ],\n         \"semantic_type\": \"\",\n
\"description\": \"\"\n       }\n    },\n    {\n      \"column\":
\"L2\",\n      \"properties\": {\n         \"dtype\": \"number\",\n
\"std\": 21.965606700596705,\n         \"min\": 9.187100410461426,\n
\"max\": 53.06760025024414,\n         \"num_unique_values\": 3,\n
\"samples\": [\n           32.954898834228516,\n
53.06760025024414,\n           9.187100410461426\n         ],\n
\"semantic_type\": \"\",\n         \"description\": \"\"\n       }\
n    },\n    {\n      \"column\": \"Linf\",\n      \"properties\": {\n
\"dtype\": \"number\",\n         \"std\": 1.0362078700237796,\n
\"min\": 0.45489999651908875,\n         \"max\": 2.4012999534606934,\n
\"num_unique_values\": 3,\n         \"samples\": [\n
0.8118000030517578,\n           2.4012999534606934,\n
0.45489999651908875\n         ],\n         \"semantic_type\": \"\",\n
\"description\": \"\"\n       }\n    }\n ]\
n}","type":"dataframe","variable_name":"nt"}
```


```
Check (numpy.linalg.norm): [2075.2666015625, 32.954898834228516,
0.8118000030517578]
```

Which norm for which purpose

- L^1 – sums every pixel's deviation: robust to a few outliers, total "ink"; basis of L1 regularisation (sparsity).
- L^2 – Euclidean length: used by k-NN distance, PCA reconstruction error and L2 weight decay; dominated by large deviations.
- L^∞ – the single worst pixel: detects a saturated/clipped pixel or the max adversarial perturbation; ignores everything else.

Since $\| x \|_\infty \leq \| x \|_2 \leq \| x \|_1 \leq \sqrt{n} \| x \|_2$, with n=4096 the L^1 value is much larger than L^2 – the spread between them shows how *spread out* (not sparse) the image energy is.

```
# Effect of each preprocessing step on the information available
orig_vals, kept = 256 * 256 * 3, IMG * IMG
print(f'Resize + grayscale : {orig_vals} -> {kept} values  (kept
{f4(kept / orig_vals * 100)} % ; colour cue lost)')
print(f'Normalise /255     : range [0,255] -> [{f4(X.min())},
{f4(X.max())}] ; mean {f4(X.mean())}, std {f4(X.std())}')
print(f'Standardise (train stats only): per-pixel mean ->
{f4(scaler_px.transform(X_tr).mean())}, std ->
{f4(scaler_px.transform(X_tr).std())}')

# Flattening: distance-based models cannot see spatial structure.
perm = rng.permutation(IMG * IMG)
d_orig = euclidean_distances(X_tr[:200], X_va[:50])
d_perm = euclidean_distances(X_tr[:200][:, perm], X_va[:50][:, perm])
print('Flattening: distances identical after randomly shuffling pixel
order ->', np.allclose(d_orig, d_perm))
print('Split: max class-proportion difference train vs test =',
f4(np.abs(split_tab.train - split_tab.test).max()))
```

```
Resize + grayscale : 196608 -> 4096 values  (kept 2.0833 % ; colour
cue lost)
Normalise /255     : range [0,255] -> [0.0, 1.0] ; mean
0.4514999985694885, std 0.15520000457763672
Standardise (train stats only): per-pixel mean -> 0.0, std -> 1.0
Flattening: distances identical after randomly shuffling pixel order -
> True
Split: max class-proportion difference train vs test = 0.0002
```

- Resizing/grayscale – removes ≈98 % of raw values; fine lesion texture and the *colour* of lesions (yellow halo vs brown spot) are lost → expected confusion between colour-defined diseases.
- Normalisation /255 and standardisation – no information lost; changes the *geometry* (feature scales) → affects conditioning (Task 4) and distance-based models.
- Flattening – the shuffled-pixel test above shows k-NN/linear models are *permutation-invariant*: spatial neighbourhood is invisible to them. A CNN uses it.

- Stratified split – keeps class proportions equal; scaler/PCA fitted on train only to avoid leakage.

Task 2 — Structure of the Feature Space (Eigen/SVD) (5 marks)

Formulas. Centred data $X_c = X - 1\mu^\top$ (n×d). Covariance $C = \frac{1}{n-1} X_c^\top X_c$. Eigendecomposition $C = Q \Lambda Q^\top$; SVD $X_c = U \Sigma V^\top$. Then $C = V \frac{\Sigma^2}{n-1} V^\top$, so

$$\lambda_i = \frac{\sigma_i^2}{n-1}, \text{EVR} = \frac{\lambda_i}{\sum_j \lambda_j}, \text{rel. reconstruction error}(k) = \frac{\sum_{i>k} \lambda_i}{\sum_j \lambda_j}.$$

Hand-worked 2×2 sub-problem

Two neighbouring pixels (p_1, p_2) observed in 4 images: $(1,2), (2,1), (3,4), (4,3)$.

1. Mean $\mu = (2.5, 2.5)$. Centred rows: $(-1.5, -0.5), (-0.5, -1.5), (0.5, 1.5), (1.5, 0.5)$.
2. $\sum p_1^2 = 2.25 + 0.25 + 0.25 + 2.25 = 5$, $\sum p_2^2 = 5$, $\sum p_1 p_2 = 0.75 \times 4 = 3$.
3. $C = \frac{1}{3}\begin{pmatrix} 5 & 3 \\ 3 & 5 \end{pmatrix} = \begin{pmatrix} 1.6667 & 1.0000 \\ 1.0000 & 1.6667 \end{pmatrix}$
4. $\det(C - \lambda I) = (1.6667 - \lambda)^2 - 1 = 0 \Rightarrow \lambda_1 = 2.6667, \lambda_2 = 0.6667$
5. λ_1: $(C - \lambda_1 I)v = 0 \Rightarrow -v_1 + v_2 = 0 \Rightarrow v_1 = 1/\sqrt{2}(1, 1) = (0.7071, 0.7071)$; λ_2: $v_2 = (0.7071, -0.7071)$
6. EVR$_1$ $= 2.6667 / 3.3333 = 0.8000 \rightarrow$ one direction ("average brightness of the pair") holds 80 % of the variance: neighbouring pixels are redundant.
7. Singular values of X_c: $\sigma_i = \sqrt{(n-1)\lambda_i} \rightarrow \sigma_1 = \sqrt{8} = 2.8284$, $\sigma_2 = \sqrt{2} = 1.4142$.

```
Xs = np.array([[1, 2], [2, 1], [3, 4], [4, 3]], float); Xs_c = Xs -
Xs.mean(0)
C2 = Xs_c.T @ Xs_c / (len(Xs) - 1)
lam2, Q2 = np.linalg.eigh(C2); s2 = np.linalg.svd(Xs_c,
compute_uv=False)
print('C =\n', C2); print('eigenvalues', lam2[::-1].round(4), '|
eigenvectors\n', Q2[:, ::-1].round(4))
print('singular values', s2.round(4), '| sigma^2/(n-1) =', (s2**2 /
3).round(4), '| EVR1 =', f4(lam2[-1] / lam2.sum()))

C =
 [[1.6667 1.    ]
 [1.     1.6667]]
eigenvalues [2.6667 0.6667] | eigenvectors
 [[ 0.7071 -0.7071]
```

```
 [ 0.7071  0.7071]]
singular values [2.8284 1.4142] | sigma^2/(n-1) = [2.6667 0.6667] |
EVR1 = 0.8
```

SVD and Eigendecomposition of the Full Training Set

Eigenvalues from singular values

$$\lambda_i = \frac{\sigma_i^2}{n-1}$$

Explained variance ratio

$$\text{EVR}_i = \frac{\lambda_i}{\sum_j \lambda_j}$$

Both routes (SVD of X_c and eighof C) are timed and compared; k for 90 / 95 / 99 % variance is reported.

```
    mu_px = X_tr.mean(0); Xc = X_tr - mu_px
t = time.time(); U, S, Vt = np.linalg.svd(Xc, full_matrices=False);
    t_svd = time.time() - t
    lam = S**2 / (len(X_tr) - 1)
t = time.time(); lam_eig = np.linalg.eigvalsh(np.cov(X_tr,
    rowvar=False))[::-1]; t_eig = time.time() - t
    evr = lam / lam.sum(); cum = np.cumsum(evr)
k90, k95, k99 = [int(np.searchsorted(cum, q) + 1) for q in (0.90, 0.95,
    0.99)]
print(f'SVD time {f4(t_svd)} s | eigh(C) time {f4(t_eig)} s') print('max
    |lambda_SVD - lambda_eig| over top 50 =', f4(np.abs(lam[:50] -
    lam_eig[:50]).max()))
print('Top-10 eigenvalues:', lam[:10].round(4))
    print('EVR of PC1..PC5:', evr[:5].round(4))
    print(f'k for 90% / 95% / 99% variance: {k90} / {k95} / {k99}  (out of
    {X.shape[1]})')
    print('lambda_1 / lambda_100 =', f4(lam[0] / lam[99]))

SVD time 86.2718 s | eigh(C) time 12.7322 s max
    |lambda_SVD - lambda_eig| over top 50 = 0.0
    Top-10 eigenvalues: [16.774   9.4144  8.6223  8.0118  2.2765  1.8403
    1.4518  1.3877  1.3336
    0.8706]
    EVR of PC1..PC5: [0.1957 0.1098 0.1006 0.0935 0.0266]
k for 90% / 95% / 99% variance: 246 / 560 / 1673  (out of 4096) lambda_1
    / lambda_100 = 228.2344970703125
```

Reconstruction Error vs Number of Components k

Projection and reconstruction

$$\hat{x} = \mu + V_k V_k^\top (x - \mu)$$

Relative reconstruction error

$$\varepsilon(k) = \frac{\| X_c - X_c V_k V_k^\top \|_F^2}{\| X_c \|_F^2} = \frac{\sum_{i>k} \lambda_i}{\sum_j \lambda_j}$$

The elbow is the point of the cumulative-EVR curve farthest from the straight line joining its ends.

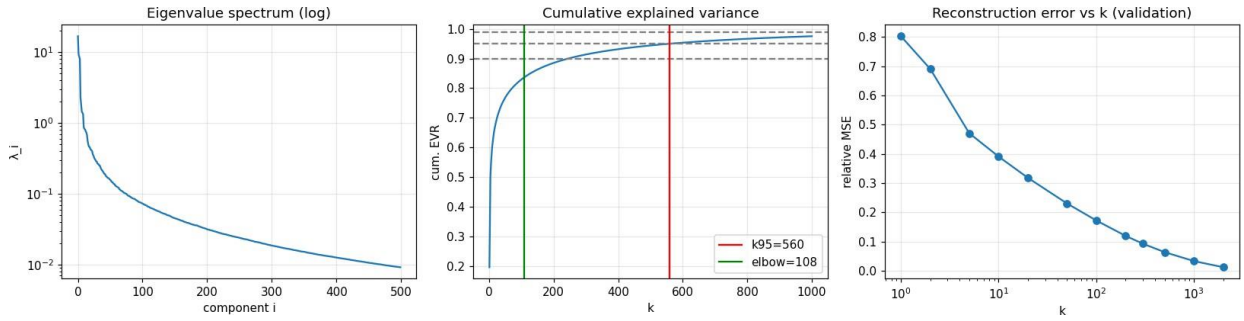
```
ks = [1, 2, 5, 10, 20, 50, 100, 200, 300, 500, 1000, 2000]
ks = [k for k in ks if k < len(S)]
Xv_c = X_va - mu_px
rec_err = [f4(np.mean((Xv_c - (Xv_c @ Vt[:k].T) @ Vt[:k])**2) /
np.mean(Xv_c**2)) for k in ks]
theory = [f4(1 - cum[k - 1]) for k in ks]
# elbow = point of the cumulative-EVR curve farthest from the chord
joining its ends (first 1000 comps)
m = min(1000, len(cum)); xs = np.arange(1, m + 1); ys_ = cum[:m]
k_elbow = int(xs[np.argmax(np.abs((ys_[-1] - ys_[0]) * xs - (m - 1) *
ys_ + m * ys_[0] - ys_[-1]))])
display(pd.DataFrame({'k': ks, 'val rel. recon error': rec_err, '1 -
cumEVR (train)': theory}))
print('Elbow k =', k_elbow, '| k95 =', k95)

fig, ax = plt.subplots(1, 3, figsize=(15, 4))
ax[0].semilogy(lam[:500]); ax[0].set(title='Eigenvalue spectrum
(log)', xlabel='component i', ylabel='λ_i')
ax[1].plot(np.arange(1, m + 1), ys_); [ax[1].axhline(q, ls='--',
c='gray') for q in (0.9, 0.95, 0.99)]
ax[1].axvline(k95, c='r', label=f'k95={k95}'); ax[1].axvline(k_elbow,
c='g', label=f'elbow={k_elbow}')
ax[1].set(title='Cumulative explained variance', xlabel='k',
ylabel='cum. EVR'); ax[1].legend()
ax[2].plot(ks, rec_err, 'o-'); ax[2].set_xscale('log');
ax[2].set(title='Reconstruction error vs k (validation)', xlabel='k',
ylabel='relative MSE')
plt.tight_layout(); plt.show()
```


```
{"summary":"{\n  \"name\": \"plt\",\n  \"rows\": 12,\n  \"fields\": [\
n    {\n      \"column\": \"k\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 597,\n        \"min\": 1,\n
\"max\": 2000,\n        \"num_unique_values\": 12,\n
\"samples\": [\n          1000,\n          500,\n          1\
n        ],\n        \"semantic_type\": \"\",\n
```

\"description\": \"\"\n rel. recon }\n },\n {\n \"column\": \"val
error\",\n \"properties\": {\n \"dtype\":
\"number\",\n\"std\": 0.2603538087494882,\n \"min\":
0.012000000104308128,\n \"max\": 0.8023999929428101,\n
\"num_unique_values\": 12,\n \"samples\": [\n
0.032999999821186066,\n 0.06369999796152115,\n
0.8023999929428101\n],\n \"semantic_type\": \"\",\n
\"description\": \"\"\n }\n },\n {\n \"column\": \"1 -
cumEVR (train)\",\n\"properties\": {\n\"dtype\":
\"number\",\n\"std\": 0.264239252094327,\n\"min\": 0.006500000134110451,\n
\"max\": 0.8043000102043152,\n
\"num_unique_values\": 12,\n \"samples\": [\n 0.05590000003576279,\n
0.024700000882148743,\n
0.8043000102043152\n],\n \"semantic_type\": \"\",\n
\"description\": \"\"\n }\n }\n]\n}","type":"dataframe"}

Elbow k = 108 | k95 = 560

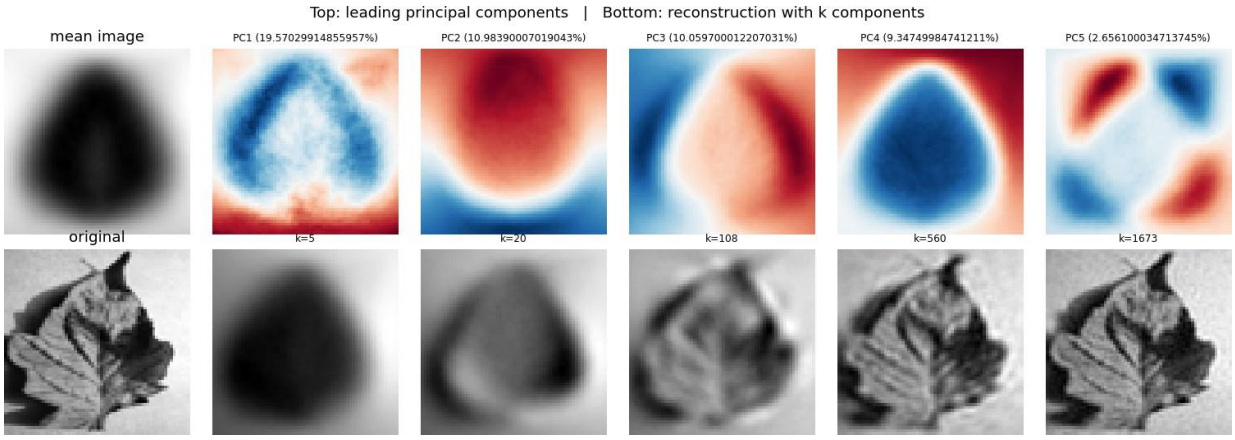


Leading Principal Components as Images

Each row of $V^\top$ is a 4,096-vector → reshaped to 64 × 64. Red/blue = positive/negative pixel weights. Bottom row: one validation image rebuilt with k components using $\hat{x}_k = \mu + V_k V_k^\top (x - \mu)$.

```
fig, ax = plt.subplots(2, 6, figsize=(14, 5))
ax[0, 0].imshow(mu_px.reshape(IMG, IMG), cmap='gray'); ax[0,
0].set_title('mean image')
for i in range(5):
    ax[0, i + 1].imshow(Vt[i].reshape(IMG, IMG), cmap='RdBu'); ax[0, i
+ 1].set_title(f'PC{i+1} ({f4(evr[i]*100)}%)', fontsize=8)
xr = X_va[0] - mu_px
ax[1, 0].imshow(X_va[0].reshape(IMG, IMG), cmap='gray'); ax[1,
0].set_title('original')
for j, k in enumerate([5, 20, k_elbow, k95, k99]):
    ax[1, j + 1].imshow((mu_px + (xr @ Vt[:k].T) @
Vt[:k]).reshape(IMG, IMG), cmap='gray'); ax[1, j +
1].set_title(f'k={k}', fontsize=8)
for a in ax.flat: a.axis('off')
```

```
plt.suptitle('Top: leading principal components   |   Bottom:
reconstruction with k components'); plt.tight_layout(); plt.show()
```



Cost of Decomposition vs Dataset Size

Thin SVD of an n × d matrix

cost=$O(nd\ min(n,d))$,eigh of C:$O(d^3)$ (independent of n¿

```
# Cost of decomposition vs dataset size: thin SVD costs O(n d
min(n,d))
cost = []
for n in [1000, 2000, 5000, 10000, len(X_tr)]:
    if n > len(X_tr): continue
    Z = X_tr[:n] - X_tr[:n].mean(0)
    t = time.time(); np.linalg.svd(Z, full_matrices=False,
compute_uv=False); cost.append((n, f4(time.time() - t)))
cost = pd.DataFrame(cost, columns=['n', 'SVD
seconds']).drop_duplicates('n'); display(cost)
```


```
{"summary":"{\n  \"name\": \"cost\",\n  \"rows\": 5,\n  \"fields\": [\
n    {\n      \"column\": \"n\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 5073,\n        \"min\":
1000,\n        \"max\": 12712,\n\"num_unique_values\": 5,\n
\"samples\": [\n          2000,\n          12712,\n          5000\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"SVD seconds\",\n
\"properties\": {\n\"dtype\": \"number\",\n                \"std\":
20.513704800644863,\n        \"min\": 0.9646,\n        \"max\":
42.2588,\n        \"num_unique_values\": 5,\n        \"samples\": [\n
4.4436,\n          42.2588,\n38.5277\n                ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n ]\n}","type":"dataframe","variable_name":"cost"}
```

Points to analyse: (i) shape of the spectrum – fast decay = strong pixel redundancy (smooth images, uniform background);

(ii) $\lambda_i = \sigma_i^2/(n-1)$ verified numerically above; (iii) choose the trade-off point (elbow vs k95) and justify it with the reconstruction table;

(iv) PC1 ≈ overall brightness/leaf-vs-background, later PCs ≈ leaf shape/orientation, *not* disease spots – variance ≠ discriminative information;

(v) cost grows ≈ linearly in n while n < d=4096, then the d² term dominates – eigh(C) is fixed O(d³) once n > d.

```
# Shared feature sets used from here on
K_PCA = k95
pca = PCA(n_components=K_PCA, random_state=SEED).fit(X_tr)
Z_tr, Z_va, Z_te = pca.transform(X_tr), pca.transform(X_va),
pca.transform(X_te)
print('PCA features:', Z_tr.shape, '| explained variance =',
f4(pca.explained_variance_ratio_.sum()))

def make_models():
    return {'LogReg (linear, parametric)':
make_pipeline(StandardScaler(), LogisticRegression(C=1.0,
max_iter=300)),
            'k-NN k=5 (non-parametric)': KNeighborsClassifier(5),
            'Random Forest (tree ensemble)':
RandomForestClassifier(150, n_jobs=-1, random_state=SEED)}
```

```
PCA features: (12712, 560) | explained variance = 0.9495000243186951
```

Task 3 — Uncertainty: Probability and Information Theory (4 marks)

Formulas (bits): $H(p) = -\sum_k p_k \log_2 p_k$, $H(p,q) = -\sum_k p_k \log_2 q_k$, $D_{KL}(p \| q) = \sum_k p_k \log_2 \frac{p_k}{q_k}$, $H(p,q) = H(p) + D_{KL}(p \| q)$.

```
def H(p): p = np.asarray(p, float); p = p[p > 0]; return -(p *
np.log2(p)).sum()
def CE(p, q, eps=1e-12): return -(np.asarray(p) * np.log2(np.clip(q,
eps, 1))).sum()
def KL(p, q, eps=1e-12):
    p, q = np.asarray(p, float), np.clip(np.asarray(q, float), eps,
1); m = p > 0
    return (p[m] * np.log2(p[m] / q[m])).sum()
```

```
p_cls = np.bincount(y_tr, minlength=K) / len(y_tr)
print('class prior p =', p_cls.round(4))
print(f'H(p) = {f4(H(p_cls))} bits | max log2(K) = {f4(np.log2(K))} |
normalised H = {f4(H(p_cls) / np.log2(K))}')
print('Majority-class accuracy =', f4(p_cls.max()), '| chance
(uniform) =', f4(1 / K))
```

```
class prior p = [0.1171 0.0551 0.1052 0.0524 0.0975 0.0924 0.0773
0.295  0.0205 0.0876]
H(p) = 3.03 bits | max log2(K) = 3.3219 | normalised H = 0.9121
Majority-class accuracy = 0.295 | chance (uniform) = 0.1
```

Cross-Entropy and KL Divergence of the Three Models

Per-image log-loss (one-hot truth, so $H(e_y)=0$)

$$H(e_y,q)=D_{KL}(e_y \| q)=-\log_2 q_y$$

Distribution level (p = validation class frequencies, $\acute{q}$ = mean predicted distribution)

$$H(p,\acute{q})=H(p)+D_{KL}(p\|\acute{q})$$

```
models_pca, rows = make_models(), []
for name, m in models_pca.items():
    m.fit(Z_tr, y_tr); Pq = m.predict_proba(Z_va)
    ce_img = np.mean(-np.log2(np.clip(Pq[np.arange(len(y_va)), y_va],
1e-12, 1)))
    p_va = np.bincount(y_va, minlength=K) / len(y_va); q_bar =
Pq.mean(0)
    rows.append([name, f4(accuracy_score(y_va, Pq.argmax(1))),
f4(ce_img), f4((Pq.max(1) < 1e-12).mean()),
                 f4(np.mean(Pq[np.arange(len(y_va)), y_va] == 0)),
f4(H(p_va)), f4(CE(p_va, q_bar)),
                 f4(KL(p_va, q_bar)), f4(KL(q_bar, p_va)),
f4(np.mean([H(r) for r in Pq]))])
info = pd.DataFrame(rows, columns=['model', 'val acc', 'per-image CE =
KL (bits)', '_', 'frac q_y = 0',
                                   'H(p_val)', 'H(p, q̀)', 'KL(p||q̀)',
'KL(q̀||p)', 'mean pred. entropy']).drop(columns='_')
display(info)
print('check H(p,q̀) = H(p) + KL(p||q̀):', np.allclose(info['H(p, q̀)'],
info['H(p_val)'] + info['KL(p||q̀)'], atol=1e-3))
```

```
{"summary":"{\n  \"name\": \"info\",\n  \"rows\": 3,\n  \"fields\": [\
n    {\n      \"column\": \"model\",\n      \"properties\": {\n
\"dtype\": \"string\",\n          \"num_unique_values\": 3,\n
\"samples\": [\n            \"LogReg (linear, parametric)\",\n
\"k-NN k=5 (non-parametric)\",\n            \"Random Forest (tree
```

```
ensemble)\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"val
acc\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.008658136828055617,\n        \"min\": 0.464,\n
\"max\": 0.4805,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.4677,\n          0.464,\n          0.4805\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"per-image CE = KL (bits)\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 5.324534984703672,\n
\"min\": 2.354,\n        \"max\": 11.7051,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          2.6171,\n
11.7051,\n          2.354\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"frac q_y = 0\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.1590601249004078,\n
\"min\": 0.0,\n        \"max\": 0.2757,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          0.0,\n
0.2757,\n          0.0004\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"H(p_val)\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.0,\n        \"min\": 3.0298,\n
\"max\": 3.0298,\n        \"num_unique_values\": 1,\n
\"samples\": [\n          3.0298\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"H(p, q\\u0304)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.09145233731294138,\n        \"min\": 3.0312,\n        \"max\":
3.1897,\n        \"num_unique_values\": 3,\n        \"samples\": [\n
3.0314\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"KL(p||q\\u0304)\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.09145233731294132,\n        \"min\":
0.0014,\n        \"max\": 0.1599,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.0016\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"KL(q\\u0304||p)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.08212821277328111,\n        \"min\": 0.0014,\n        \"max\":
0.1438,\n        \"num_unique_values\": 3,\n        \"samples\": [\n
0.0017\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"mean pred. entropy\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.9280954207407771,\n        \"min\":
0.942,\n        \"max\": 2.7957,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          1.7856\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n ]\n}","type":"dataframe","variable_name":"info"}
```


```
check H(p,q) = H(p) + KL(p||q): True
```

KL asymmetry with my own numbers

p=[0.5, 0.3, 0.2) (true), q=[0.8, 0.15, 0.05) (over-confident model)

$$D_{KL}(p \parallel q) = 0.5 \log_2 \frac{0.5}{0.8} + 0.3 \log_2 \frac{0.3}{0.15} + 0.2 \log_2 \frac{0.2}{0.05} = -0.3390 + 0.3000 + 0.4000 = 0.3610 \text{ bits}$$

$$D_{KL}(q \parallel p) = 0.8 \log_2 \frac{0.8}{0.5} + 0.15 \log_2 \frac{0.15}{0.3} + 0.05 \log_2 \frac{0.05}{0.2} = 0.5425 - 0.1500 - 0.1000 = 0.2925 \text{ bits}$$

- $DKL(p \parallel q)$ weights by the truth: it punishes the model for giving low probability where real cases occur (the rare class 3 dominates: 0.4 bits) → "mass-covering", what training with cross-entropy minimises.
- $DKL(q \parallel p)$ weights by the model: it punishes confident predictions in places the truth doesn't support → "mode-seeking".

```
p3, q3 = np.array([0.5, 0.3, 0.2]), np.array([0.8, 0.15, 0.05])
print('KL(p||q) =', f4(KL(p3, q3)), '| KL(q||p) =', f4(KL(q3, p3)))

KL(p||q) = 0.361 | KL(q||p) = 0.2925
```

Why minimising CE ≡ minimising KL: $H(p, q_\theta) = H(p) + D_{KL}(p \parallel q_\theta)$ and $H(p)$ does not depend on θ, so $\nabla_\theta H(p, q_\theta) = \nabla_\theta D_{KL}(p \parallel q_\theta)$ and both have the same minimiser.

Points to analyse: k-NN gives hard 0/1-ish probabilities → some $q_y = 0$ → huge CE (over-confident errors); LR is calibrated-ish → lower CE even if accuracy is similar; RF averages trees → soft probabilities, high predictive entropy.

Task 4 — Numerical Computation Issues (3 marks)

Softmax: $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$. Stable form: with $m = max_j\, z_j$, $\text{softmax}(z)_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}$ (multiply numerator and denominator by e^{-m} → identical value). Log-sum-exp: $\log \sum_j e^{z_j} = m + \log \sum_j e^{z_j - m}$, so $\log \text{softmax}(z)_i = z_i - m - \log \sum_j e^{z_j - m}$.

Hand check for z=[1000, 999, 995): $z - m$=[0, −1, −5), e^{z-m}=[1, 0.3679, 0.0067), sum ¿ 1.3746 * softmax ¿ [0.7275, 0.2676, 0.0049). Naively e^{1000} exceeds float64's max ($\approx e^{709.78}$) → inf/inf = nan.

```
def softmax_naive(z): e = np.exp(z); return e / e.sum()
def softmax_stable(z): e = np.exp(z - np.max(z)); return e / e.sum()
def log_softmax(z): m = np.max(z); return z - m - np.log(np.exp(z -
m).sum())

with np.errstate(all='ignore'):
    for z in [np.array([1000., 999, 995]), np.array([-1000., -999, -
```

```
995]), np.array([3., 2, -2])]:
        print(f'z={z}: naive={softmax_naive(z)} |
stable={softmax_stable(z).round(4)} | log-
softmax={log_softmax(z).round(4)}')
    print('float32 overflow threshold: exp(88.8) =',
np.exp(np.float32(88.8)), '| float64: exp(709.8) =', np.exp(709.8))
    z = np.array([3., 2, -2]); print('naive == stable for moderate
z:', np.allclose(softmax_naive(z), softmax_stable(z)))
```

```
z=[1000.  999.  995.]: naive=[nan nan nan] | stable=[0.7275 0.2676
0.0049] | log-softmax=[-0.3182 -1.3182 -5.3182]
z=[-1000.  -999.  -995.]: naive=[nan nan nan] | stable=[0.0066 0.0179
0.9756] | log-softmax=[-5.0247 -4.0247 -0.0247]
z=[ 3.  2. -2.]: naive=[0.7275 0.2676 0.0049] | stable=[0.7275 0.2676
0.0049] | log-softmax=[-0.3182 -1.3182 -5.3182]
float32 overflow threshold: exp(88.8) = inf | float64: exp(709.8) =
inf
naive == stable for moderate z: True
```

Condition Number of the Feature Covariance

Condition number

$$\kappa(C)=\frac{\lambda_{max}}{\lambda_{min}}$$

GD contraction per step (quadratic loss, best fixed step)

$$\| w_t - w^{\dot{c}} \| \leq \left(\frac{\kappa-1}{\kappa+1}\right)^{t} \| w_0 - w^{\dot{c}} \|$$

```
# Conditioning of the feature covariance
tol = lam[0] * 1e-10
num_rank = int((lam > tol).sum())
kappa_raw = lam[0] / lam[num_rank - 1]
Zs = StandardScaler().fit_transform(Z_tr)
lam_z = np.linalg.eigvalsh(np.cov(Z_tr, rowvar=False))[::-1]
cond = pd.DataFrame({'features': ['raw pixels (centred)', f'PCA-
{K_PCA} scores', f'PCA-{K_PCA} standardised (whitened)'],
                     'lambda_max': [f4(lam[0]), f4(lam_z[0]),
f4(np.linalg.eigvalsh(np.cov(Zs, rowvar=False)).max())],
                     'lambda_min (non-zero)': [lam[num_rank - 1],
f4(lam_z[-1]), f4(np.linalg.eigvalsh(np.cov(Zs,
rowvar=False)).min())]})
cond['condition number κ'] = cond['lambda_max'] / cond['lambda_min
(non-zero)']
cond['GD contraction (κ-1)/(κ+1)'] = (cond['condition number κ'] - 1)
/ (cond['condition number κ'] + 1)
```

```
display(cond); print('numerical rank of raw covariance:', num_rank,
'of', X.shape[1])
```


```
{"summary":"{\n  \"name\": \"cond\",\n  \"rows\": 3,\n  \"fields\": [\
n    {\n      \"column\": \"features\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 3,\n
\"samples\": [\n          \"raw pixels (centred)\",\n          \"PCA-
560 scores\",\n          \"PCA-560 standardised (whitened)\"\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"lambda_max\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 9.090899652086092,\n        \"min\":
1.0281,\n        \"max\": 16.77400016784668,\n
\"num_unique_values\": 3,\n        \"samples\": [\n
16.77400016784668,\n          16.774,\n          1.0281\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"lambda_min (non-zero)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.5695019386504898,\n        \"min\": 2.441189280943945e-05,\n
\"max\": 0.9899,\n        \"num_unique_values\": 3,\n
\"samples\": [\n2.441189280943945e-05,\n                    0.007,\n
0.9899\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"condition number \\u03ba\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 396021.0539837204,\n
\"min\": 1.0385897565410647,\n        \"max\": 687124.1119558171,\n
\"num_unique_values\": 3,\n        \"samples\": [\n
687124.1119558171,\n          2396.285714285714,\n
1.0385897565410647\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"GD
contraction (\\u03ba-1)/(\\u03ba+1)\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.5661797176776501,\n
\"min\": 0.018929633300297285,\n        \"max\": 0.9999970893219223,\n
\"num_unique_values\": 3,\n        \"samples\": [\n
0.9999970893219223,\n          0.9991657231392647,\n
0.018929633300297285\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n ]\
n}","type":"dataframe","variable_name":"cond"}
```


```
numerical rank of raw covariance: 4096 of 4096
```

For a quadratic loss with Hessian eigenvalues λ_{max}, λ_{min}, GD with the best fixed step converges as $\| wt - w¿ \| \leq \left(\frac{\kappa - 1}{\kappa + 1}\right)t\| w0 - w¿ \|$, $\kappa = \lambda_{max}/\lambda_{min}$, and is stable only for $\eta \in 2/\lambda_{max}$. Huge κ → the step size is capped by the steep direction while the flat directions barely move → slow, zig-zagging training (tested in Task 8).

Task 5 — Differentiate Baseline Learning Strategies (4 marks)

Model	Hypothesis space	Inductive bias
Multinomial logistic regression	Linear decision boundaries $arg\ max_k\ w_k^T\ x + b_k$	classes linearly separable; smooth, global; L2 → small weights
k-NN (k=5)	Any boundary, piecewise (Voronoi)	local constancy: nearby in Euclidean pixel space ⇒ same class
Random forest	Axis-aligned piecewise-constant regions, averaged	individual features (pixels) carry thresholdable information; bagging reduces variance

```
def run(model, Xa, ya, Xb, yb):
    t = time.time(); model.fit(Xa, ya); tr = time.time() - t
    t = time.time(); pr = model.predict(Xb); inf = (time.time() - t) /
len(Xb) * 1000
    return f4(accuracy_score(yb, pr)),
f4(precision_recall_fscore_support(yb, pr, average='macro',
zero_division=0)[2]), f4(tr), f4(inf)

rows = []
for feat, (A, B) in {'raw 4096': (X_tr, X_va), f'PCA-{K_PCA}': (Z_tr,
Z_va)}.items():
    for name, m in make_models().items():
        if feat == 'raw 4096' and name.startswith('k-NN'):
            m = make_pipeline(StandardScaler(), m)
        rows.append([name, feat, *run(m, A, y_tr, B, y_va)])
base = pd.DataFrame(rows, columns=['model', 'features', 'val acc',
'val macro-F1', 'train s', 'infer ms/img'])
display(base.pivot(index='model', columns='features', values=['val
acc', 'train s']))
```


```
{"summary":"{\n  \"name\": \"display(base\",\n  \"rows\": 3,\n
\"fields\": [\n    {\n      \"column\": [\n        \"model\",\n
\"\"\n      ],\n      \"properties\": {\n        \"dtype\":
\"string\",\n        \"num_unique_values\": 3,\n        \"samples\":
[\n          \"LogReg (linear, parametric)\",\n          \"Random
Forest (tree ensemble)\",\n          \"k-NN k=5 (non-parametric)\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": [\n        \"val acc\",\n
\"PCA-560\"\n      ],\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.008658136828055617,\n        \"min\":
0.464,\n        \"max\": 0.4805,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          0.4677,\n          0.4805,\n          0.464\
n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": [\n
```

\"val acc\",\n \"raw 4096\"\n],\n \"properties\": {\n\"dtype\": \"number\",\n\"std\": 0.1549733633026441,\ n\"min\": 0.3634,\n\"max\": 0.6648,\n\"num_unique_values\": 3,\n\"samples\": [\n0.3634,\n

0.6648,\n 0.4515\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": [\n \"train s\",\n \"PCA-560\"\n],\n \"properties\": {\n\"dtype\": \"number\",\n\"std\": 32.70167201984224,\n\"min\": 0.0054,\n\"max\": 57.5828,\n\"num_unique_values\": 3,\n\"samples\": [\n 1.9272,\n57.5828,\n0.0054\n],\n \"semantic_type\": \"\",\n\"description\": \"\"\n}\ n},\n{\n\"column\": [\n\"train s\",\n

\"raw 4096\"\n],\n \"properties\": {\n \"dtype\": \"number\",\n \"std\": 47.8551526668759,\n \"min\": 0.7501,\n\"max\": 89.111,\n\"num_unique_values\": 3,\n \"samples\": [\n89.111,\n76.7835,\n 0.7501\n],\n\"semantic_type\": \"\",\n \"description\": \"\"\n}\n}\n]\n}","type":"dataframe"}

Points to analyse: which model wins on raw vs PCA and why – e.g. k-NN benefits from PCA (denoises distances, fewer irrelevant dims); RF may lose accuracy after PCA because PCs mix pixels and destroy the axis-aligned structure it splits on; LR changes little because PCA is a rotation + truncation of a space it is already linear in, but trains much faster.

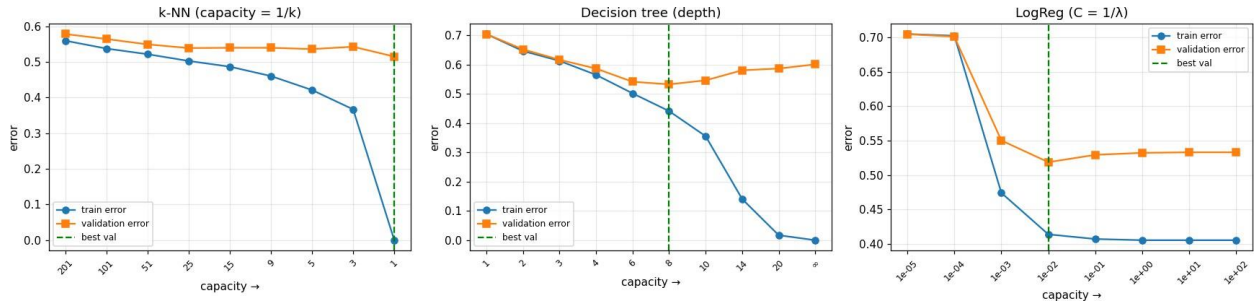
Task 6 — Capacity, Overfitting and Underfitting (4 marks)

Error ¿ $1 -$ accuracy. Generalisation gap ¿ $E_{val} - E_{train}$. Capacity increases with: tree depth ↑, $1/k$ ↑ (k-NN), $C = 1/\lambda$ ↑ (LR). All curves on PCA features, evaluated on the validation set.

```
def curve(make, grid):
    tr, va = [], []
    for g in grid:
        m = make(g).fit(Z_tr, y_tr)
        tr.append(1 - accuracy_score(y_tr, m.predict(Z_tr)));
va.append(1 - accuracy_score(y_va, m.predict(Z_va)))
    return np.array(tr), np.array(va)

fam = {
  'k-NN (capacity = 1/k)': (lambda k: KNeighborsClassifier(k), [201,
101, 51, 25, 15, 9, 5, 3, 1]),
  'Decision tree (depth)': (lambda d:
DecisionTreeClassifier(max_depth=d, random_state=SEED), [1, 2, 3, 4,
6, 8, 10, 14, 20, None]),
  'LogReg (C = 1/λ)': (lambda c: make_pipeline(StandardScaler(),
LogisticRegression(C=c, max_iter=300)), list(np.logspace(-5, 2, 8))),
}
curves = {}
fig, ax = plt.subplots(1, 3, figsize=(16, 4))
```

```
for a, (nm, (mk, grid)) in zip(ax, fam.items()):
    tr, va = curve(mk, grid); curves[nm] = (grid, tr, va)
    xs = np.arange(len(grid)); a.plot(xs, tr, 'o-', label='train
error'); a.plot(xs, va, 's-', label='validation error')
    a.set_xticks(xs); a.set_xticklabels([('∞' if g is None else
(f'{g:.0e}' if isinstance(g, float) else g)) for g in grid],
rotation=45, fontsize=8)
    a.axvline(np.argmin(va), c='g', ls='--', label='best val');
a.set(title=nm, xlabel='capacity →', ylabel='error');
a.legend(fontsize=8)
plt.tight_layout(); plt.show()
display(pd.DataFrame({nm: {'best setting': g[np.argmin(v)], 'train
err': f4(t[np.argmin(v)]), 'val err': f4(v.min()),
                           'gap at best': f4(v.min() -
t[np.argmin(v)]), 'gap at max capacity': f4(v[-1] - t[-1])}
                      for nm, (g, t, v) in curves.items()}))
```

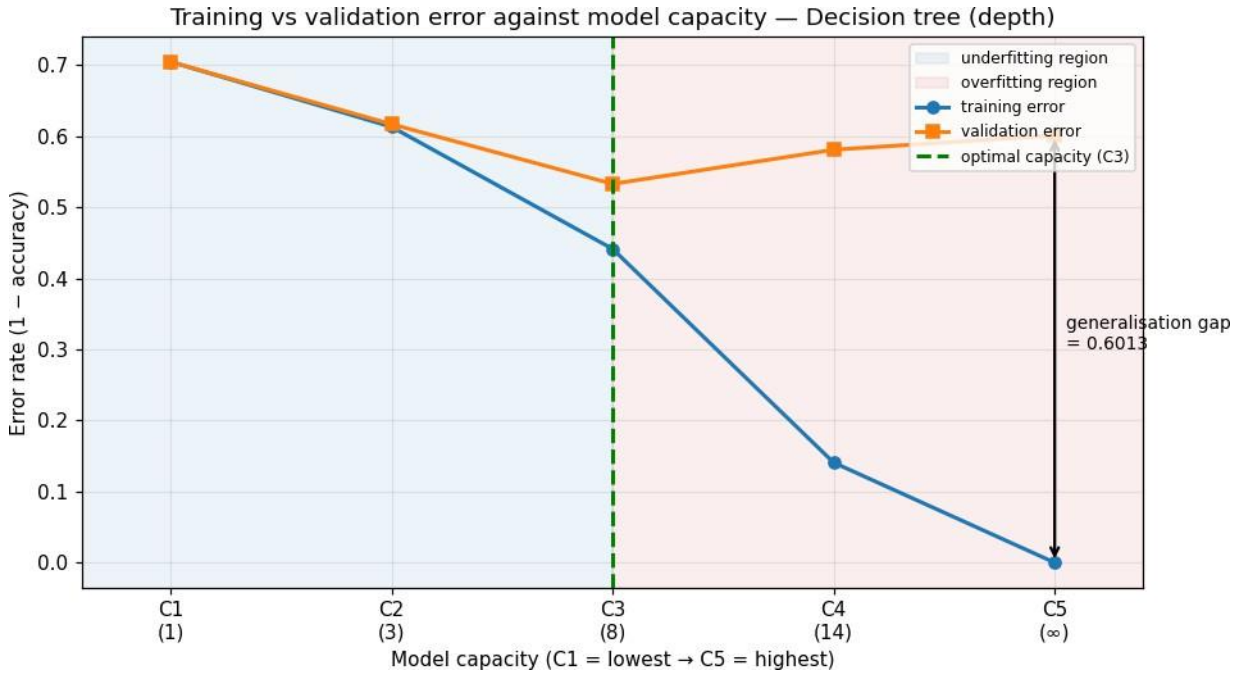



```
{"summary":"{\n \"name\": \"                                                    for nm, (g, t, v)
in curves\",\n \"rows\": 5,\n \"fields\": [\n                                {\n
\"column\": \"k-NN (capacity = 1/k)\",\n\"properties\": {\n
\"dtype\": \"number\",\n\"std\": 0.35365011381307376,\n
\"min\": 0.0,\n\"max\": 1.0,\n\"num_unique_values\":
3,\n                \"samples\": [\n                        1.0,\n                0.0,\n
0.5151\n                ],\n                \"semantic_type\": \"\",\n
\"description\": \"\"\n}\n},\n{\n\"column\":
\"Decision tree (depth)\",\n\"properties\": {\n
\"dtype\": \"number\",\n\"std\": 3.396958527418314,\n
\"min\": 0.0915,\n\"max\": 8.0,\n
\"num_unique_values\": 5,\n\"samples\": [\n0.4416,\n
0.6013,\n                0.533\n                ],\n                \"semantic_type\":
\"\",\n                \"description\": \"\"\n                }\n        },\n        {\n
\"column\": \"LogReg (C = 1/\\u03bb)\",\n\"properties\": {\n
\"dtype\": \"number\",\n                        \"std\": 0.21880240629389797,\n
\"min\": 0.01,\n                \"max\": 0.5187,\n
\"num_unique_values\": 5,\n\"samples\": [\n0.4138,\n 0.1278,\n0.5187\n],\n\"semantic_type\":
\"\",\n\"description\": \"\"\n}\n ]\ n}","type":"dataframe"}
```

Annotated capacity plot (C1 → C5)

```
# Pick the family whose optimum is most interior and sample 5 capacity
levels around it
def interior(v): o = int(np.argmin(v)); return min(o, len(v) - 1 - o)
fam_name = max(curves, key=lambda n: interior(curves[n][2]))
grid, tr, va = curves[fam_name]; o = int(np.argmin(va)); L = len(grid)
- 1
sel = sorted(set([0, o // 2, o, o + (L - o + 1) // 2, L]))
while len(sel) < 5: sel = sorted(set(sel + [next(i for i in range(L +
1) if i not in sel)]))
sel = sel[:5]
labels = [f'C{i+1}\n({"∞" if grid[j] is None else (f"{grid[j]:.0e}" if
isinstance(grid[j], float) else grid[j])})' for i, j in
enumerate(sel)]
tr5, va5 = tr[sel], va[sel]; opt = int(np.argmin(va5)); x5 =
np.arange(1, 6)

fig, a = plt.subplots(figsize=(9, 5))
a.axvspan(0.6, x5[opt], color='tab:blue', alpha=.08,
label='underfitting region')
a.axvspan(x5[opt], 5.4, color='tab:red', alpha=.08, label='overfitting
region')
a.plot(x5, tr5, 'o-', lw=2, label='training error'); a.plot(x5, va5,
's-', lw=2, label='validation error')
a.axvline(x5[opt], c='g', ls='--', lw=2, label=f'optimal capacity
({labels[opt].split()[0]})')
g = int(np.argmax(va5 - tr5))
a.annotate('', xy=(x5[g], tr5[g]), xytext=(x5[g], va5[g]),
arrowprops=dict(arrowstyle='<->', color='k', lw=1.5))
a.text(x5[g] + 0.05, (tr5[g] + va5[g]) / 2, f'generalisation gap\n=
{f4(va5[g] - tr5[g])}', fontsize=9)
a.set_xticks(x5); a.set_xticklabels(labels); a.set_xlim(0.6, 5.4)
a.set(title=f'Training vs validation error against model capacity —
{fam_name}', xlabel='Model capacity (C1 = lowest → C5 = highest)',
ylabel='Error rate (1 − accuracy)')
a.legend(loc='upper right', fontsize=8); plt.tight_layout();
plt.show()
print(pd.DataFrame({'capacity': [l.replace('\n', ' ') for l in
labels], 'train err': tr5.round(4), 'val err': va5.round(4), 'gap':
(va5 - tr5).round(4)}))
```



capacity	train err	val err	gap
0C1 (1)	0.7050	0.7048	-0.0002
1C2 (3)	0.6134	0.6171	0.0037
2C3 (8)	0.4416	0.5330	0.0915
3 C4 (14)	0.1408	0.5811	0.4403
4C5 (∞)	0.0000	0.6013	0.6013

Causes: *underfitting* (left) – hypothesis too simple to represent the class boundaries, both errors high, gap small (high bias); *overfitting* (right) – model fits pixel noise / individual images (k=1, unlimited depth → train error ≈ 0), gap large (high variance).

Test-set selection bias

```
# Choose the best of all configurations on half of the test set (A),
then report on A vs the untouched half (B)
iA, iB = train_test_split(np.arange(len(y_te)), test_size=0.5,
stratify=y_te, random_state=SEED)
res = []
for nm, (mk, grid) in fam.items():
    for gval in grid:
        p = mk(gval).fit(Z_tr, y_tr).predict(Z_te); res.append((nm,
gval, accuracy_score(y_te[iA], p[iA]), accuracy_score(y_te[iB],
p[iB])))
best = max(res, key=lambda r_: r_[2])
cv = cross_val_score(KNeighborsClassifier(5), Z_tr, y_tr,
cv=StratifiedKFold(5, shuffle=True, random_state=SEED))
print(f'selected on test-half A: {best[0]} = {best[1]}')
print(f'  accuracy reported on A (biased) = {f4(best[2])} | on fresh
```

```
half B (unbiased) = {f4(best[3])} | optimism = {f4(best[2] -
best[3])}')
print(f'5-fold CV accuracy of k-NN(k=5) on train: {f4(cv.mean())} ±
{f4(cv.std())}')
```

```
selected on test-half A: k-NN (capacity = 1/k) = 1
  accuracy reported on A (biased) = 0.4949 | on fresh half B
(unbiased) = 0.4743 | optimism = 0.0206
5-fold CV accuracy of k-NN(k=5) on train: 0.464 ± 0.0073
```

Choosing the max over many configurations on the same data you report on picks up the configuration that was *luckiest* on that sample → the reported error is optimistically biased (selection effect, "winner's curse"). A validation set or k-fold CV makes selection on data that is independent of the final test estimate.

□ What this means for AgriVision *(write 1–2 lines in your own words after running)*:

Task 7 — Bias–Variance Decomposition and MLE vs MAP (4 marks)

Bootstrap: train B models $\hat{f}_b$ on resamples of the training set; $\bar{f}(x)=\frac{1}{B}\sum_b \hat{f}_b(x)$. Using squared error on probability vectors (Brier) with one-hot target e_y:

$$1/B\sum_b \|\hat{f}_b(x)-e_y\|^2 = \|\bar{f}(x)-e_y\|^2 + 1/B\sum_b \|\hat{f}_b(x)-\bar{f}(x)\|^2$$

With observed labels the irreducible noise cannot be separated and sits inside the bias term.

```
B = 10 if TEST_MODE else 20
n_bs = min(3000, len(Z_tr)); n_ev = min(1000, len(Z_va))
Zb, yb_, Ze, ye = Z_tr[:n_bs], y_tr[:n_bs], Z_va[:n_ev], y_va[:n_ev]
E = np.eye(K)[ye]
def bv(make):
    P, preds = [], []
    for b in range(B):
        i = rng.integers(0, n_bs, n_bs); m = make().fit(Zb[i], yb_[i])
        pp = np.zeros((n_ev, K)); pp[:, m.classes_] =
m.predict_proba(Ze); P.append(pp); preds.append(pp.argmax(1))
    P, preds = np.array(P), np.array(preds); Pbar = P.mean(0)
    err = ((P - E)**2).sum(2).mean(); bias2 = ((Pbar -
E)**2).sum(1).mean(); var = ((P - Pbar)**2).sum(2).mean()
    main = np.array([np.bincount(c, minlength=K).argmax() for c in
preds.T])
    return dict(err=f4(err), bias2=f4(bias2), var=f4(var),
sum=f4(bias2 + var),
                err01=f4((preds != ye).mean()), bias01=f4((main !=
ye).mean()), var01=f4((preds != main).mean()))
bvt = pd.DataFrame({'low capacity: tree depth 2': bv(lambda:
```

```
DecisionTreeClassifier(max_depth=2, random_state=SEED)),
                    'high capacity: tree depth ∞': bv(lambda:
DecisionTreeClassifier(random_state=SEED))}).T
bvt.columns = ['Brier error', 'bias²', 'variance', 'bias²+var', '0-1
error', '0-1 bias', '0-1 variance']
display(bvt)
```


```
{"summary":"{\n  \"name\": \"bvt\",\n  \"rows\": 2,\n  \"fields\": [\n
{\n      \"column\": \"Brier error\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.4084955874914684,\n
\"min\": 0.8061,\n        \"max\": 1.3838,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          1.3838,\n
0.8061\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"bias\\u00b2\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.06703372285648478,\n        \"min\":
0.7001,\n        \"max\": 0.7949,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          0.7001,\n          0.7949\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"variance\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.47560002102607185,\n        \"min\": 0.0111,\n        \"max\":
0.6837,\n        \"num_unique_values\": 2,\n        \"samples\": [\n
0.6837,\n          0.0111\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"bias\\u00b2+var\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.4084955874914684,\n
\"min\": 0.8061,\n        \"max\": 1.3838,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          1.3838,\n
0.8061\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"0-1
error\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.009475230867899713,\n        \"min\": 0.6785,\n
\"max\": 0.6919,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          0.6919,\n          0.6785\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"0-1 bias\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.09828784258493012,\n        \"min\": 0.543,\n        \"max\":
0.682,\n        \"num_unique_values\": 2,\n        \"samples\": [\n
0.543,\n          0.682\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"0-1 variance\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.23440589796334046,\n
\"min\": 0.2142,\n        \"max\": 0.5457,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          0.5457,\n
0.2142\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n ]\
n}","type":"dataframe","variable_name":"bvt"}
```

Check with Known Noise (Regression)

$y=\sin(2\pi x)+\varepsilon, \varepsilon \sim N(0, \sigma^2)$, $\sigma^2=0.09$. Over 200 fresh training sets:

$$E\left[(y-\hat{f}(x))2\right]=¿$$

```
# Check against a case where noise is KNOWN: y = sin(2πx) + ε, σ² =
0.09 (fresh training sets, true expectation)
sig = 0.3; xt = np.linspace(0, 1, 50); ft = np.sin(2 * np.pi * xt);
out = []
for deg in (1, 3, 9):
    preds = []
    for _ in range(200):
        xx = rng.uniform(0, 1, 30); yy = np.sin(2 * np.pi * xx) +
rng.normal(0, sig, 30)
        preds.append(np.polynomial.Polynomial.fit(xx, yy, deg)(xt))
    preds = np.array(preds); yt = ft + rng.normal(0, sig, (200, 50))
    out.append([deg, f4(((preds - yt)**2).mean()), f4(((preds.mean(0)
- ft)**2).mean()), f4(preds.var(0).mean()), sig**2])
kn = pd.DataFrame(out, columns=['poly degree', 'measured MSE',
'bias²', 'variance', 'σ²'])
kn['bias²+var+σ²'] = kn['bias²'] + kn['variance'] + kn['σ²'];
display(kn)
```


```
{"summary":"{\n  \"name\": \"kn\",\n  \"rows\": 3,\n  \"fields\": [\n
{\n      \"column\": \"poly degree\",\n      \"properties\": {\n
\"dtype\": \"number\",\n         \"std\": 4,\n         \"min\": 1,\n
\"max\": 9,\n\"num_unique_values\": 3,\n            \"samples\":
[\n          1,\n          3,\n          9\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"measured MSE\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
145.41055396524695,\n         \"min\": 0.1142,\n         \"max\":
252.079,\n         \"num_unique_values\": 3,\n         \"samples\": [\n
0.327,\n          0.1142,\n  252.079\n                ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"bias\\u00b2\",\n
\"properties\": {\n\"dtype\": \"number\",\n                \"std\":
0.7566664809104031,\n         \"min\": 0.0054,\n         \"max\":
1.4069,\n         \"num_unique_values\": 3,\n         \"samples\": [\n
0.2117,\n          0.0054,\n          1.4069\n         ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"variance\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
144.67797304131452,\n         \"min\": 0.0174,\n         \"max\":
250.6097,\n\"num_unique_values\": 3,\n\"samples\":        [\n
0.0228,\n          0.0174,\n          250.6097\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"\\u03c3\\u00b2\",\n
```

\"properties\": {\n\"dtype\": \"number\",\n 1.6996749443881478e-17,\n\"min\": 0.09,\n \"std\": \"max\": 0.09,\n\"num_unique_values\": 1,\n\"samples\": [\n 0.09\n],\n\"semantic_type\": \"\",\n \"description\": \"\"\n}\n},\n{\n\"column\": \"bias\\u00b2+var+\\u03c3\\u00b2\",\n\"properties\": {\n

\"dtype\": \"number\",\n \"std\": 145.4276142597753,\n \"min\": 0.1128,\n \"max\": 252.10660000000001,\n \"num_unique_values\": 3,\n\"samples\": [\n0.3245\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n }\n]\n}","type":"dataframe","variable_name":"kn"}

Points to analyse: depth-2 tree = bias-dominated, depth-∞ = variance-dominated (compare with Task 6 curves). The Brier identity is exact; for 0-1 loss the error is *not* bias + variance (variance can even help when the main prediction is wrong) – that is the gap. In the known-noise check the gap comes only from finite replicas (Monte-Carlo error).

MLE vs MAP for a class proportion (Beta prior)

$x_i \sim \text{Bernoulli}(\theta)$, $k = \sum x_i$, prior $\theta \sim \text{Beta}(\alpha, \beta)$:

$$\hat{\theta}_{MLE} = \frac{k}{n}, \quad \hat{\theta}_{MAP} = \frac{k+\alpha-1}{n+\alpha+\beta-2} = w\,\theta_{prior\ mode} + (1-w)\,\hat{\theta}_{MLE}, \quad w = \frac{\alpha+\beta-2}{n+\alpha+\beta-2}$$

Prior chosen: Beta(5, 45) (agronomist's belief: ~10 % of photos are healthy leaves; mode $4/48 = 0.0833$).

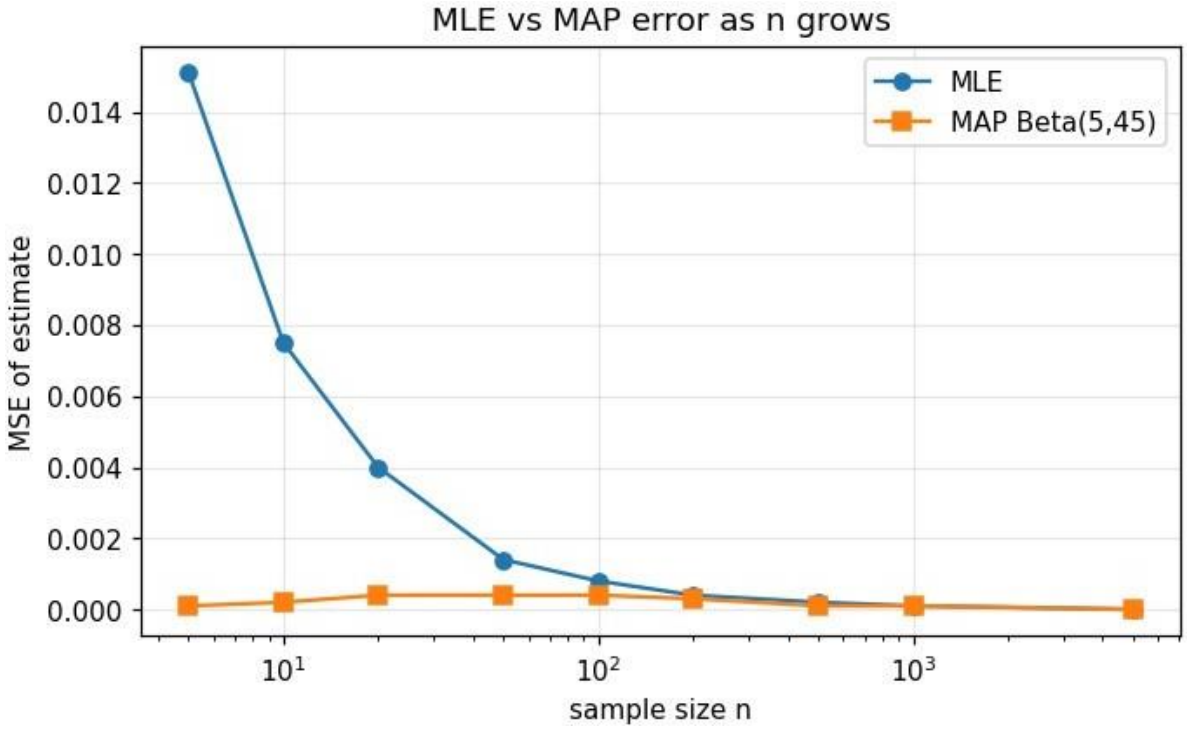
```
hc = CLASSES.index('healthy') if 'healthy' in CLASSES else K - 1
a_, b_ = 5, 45; theta_true = (y == hc).mean()
s = rng.permutation(len(y))[:20]; k20 = int((y[s] == hc).sum())
print(f'class = {CLASSES[hc]}, full-data proportion =
{f4(theta_true)}')
print(f'Worked example n=20: k={k20}  MLE = {k20}/20 = {f4(k20/20)}
MAP = ({k20}+4)/(20+48) = {f4((k20+a_-1)/(20+a_+b_-2))}')
ns = [n for n in [5, 10, 20, 50, 100, 200, 500, 1000, 5000] if n <
len(y)]; rows = []
for n in ns:
    ks_ = np.array([(y[rng.choice(len(y), n, replace=False)] ==
hc).sum() for _ in range(200)])
    mle, mapp = ks_ / n, (ks_ + a_ - 1) / (n + a_ + b_ - 2)
    rows.append([n, f4(mle.mean()), f4(mle.std()), f4(mapp.mean()),
f4(mapp.std()), f4((a_ + b_ - 2) / (n + a_ + b_ - 2)),
                 f4(np.mean((mle - theta_true)**2)), f4(np.mean((mapp
- theta_true)**2))])
mm = pd.DataFrame(rows, columns=['n', 'MLE mean', 'MLE sd', 'MAP
mean', 'MAP sd', 'prior weight w', 'MSE MLE', 'MSE MAP']); display(mm)
plt.figure(figsize=(7, 4)); plt.semilogx(ns, mm['MSE MLE'], 'o-',
label='MLE'); plt.semilogx(ns, mm['MSE MAP'], 's-', label='MAP
Beta(5,45)')
```

```
plt.xlabel('sample size n'); plt.ylabel('MSE of estimate');
plt.title('MLE vs MAP error as n grows'); plt.legend(); plt.show()

class = healthy, full-data proportion = 0.0876
Worked example n=20: k=1  MLE = 1/20 = 0.05  MAP = (1+4)/(20+48) =
0.0735

{"summary":"{\n  \"name\": \"mm\",\n  \"rows\": 9,\n  \"fields\": [\n
{\n      \"column\": \"n\",\n      \"properties\": {\n
\"dtype\": \"number\",\n         \"std\": 1621,\n         \"min\": 5,\n
\"max\": 5000,\n         \"num_unique_values\": 9,\n
\"samples\": [\n           1000,\n           10,\n           200\n
],\n         \"semantic_type\": \"\",\n         \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"MLE mean\",\n
\"properties\": {\n         \"dtype\": \"number\",\n         \"std\":
0.002033469940766274,\n         \"min\": 0.084,\n         \"max\":
0.09,\n         \"num_unique_values\": 8,\n         \"samples\": [\n
0.084,\n           0.0855,\n           0.09\n         ],\n
\"semantic_type\": \"\",\n         \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"MLE sd\",\n      \"properties\":
{\n         \"dtype\": \"number\",\n         \"std\":
0.04047660160855624,\n         \"min\": 0.0036,\n         \"max\":
0.1229,\n         \"num_unique_values\": 9,\n         \"samples\": [\n
0.0085,\n           0.0863,\n           0.0199\n         ],\n
\"semantic_type\": \"\",\n         \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"MAP mean\",\n      \"properties\":
{\n         \"dtype\": \"number\",\n         \"std\":
0.0016241236953439767,\n         \"min\": 0.0834,\n         \"max\":
0.0878,\n         \"num_unique_values\": 9,\n         \"samples\": [\n
0.0878,\n           0.0834,\n           0.0847\n         ],\n
\"semantic_type\": \"\",\n         \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"MAP sd\",\n      \"properties\":
{\n         \"dtype\": \"number\",\n         \"std\":
0.00551651560719667,\n         \"min\": 0.0035,\n         \"max\":
0.0196,\n         \"num_unique_values\": 9,\n         \"samples\": [\n
0.0082,\n           0.0149,\n           0.0161\n         ],\n
\"semantic_type\": \"\",\n         \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"prior weight w\",\n
\"properties\": {\n\"dtype\": \"number\",\n                \"std\":
0.3471519303961564,\n         \"min\": 0.0095,\n         \"max\":
0.9057,\n         \"num_unique_values\": 9,\n         \"samples\": [\n
0.0458,\n           0.8276,\n           0.1935\n         ],\n
\"semantic_type\": \"\",\n         \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"MSE MLE\",\n      \"properties\":
{\n         \"dtype\": \"number\",\n         \"std\":
0.005079069249817771,\n         \"min\": 0.0,\n         \"max\":
0.0151,\n         \"num_unique_values\": 9,\n         \"samples\": [\n
0.0001,\n           0.0075,\n           0.0004\n         ],\n
\"semantic_type\": \"\",\n         \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"MSE MAP\",\n      \"properties\":
```

{\n \"dtype\": \"number\",\n \"std\": 0.00015634719199411433,\n\"min\": 0.0,\n\"max\": 0.0004,\n\"num_unique_values\": 5,\n\"samples\": [\n 0.0002,\n0.0,\n0.0004\n],\n \"semantic_type\": \"\",\n\"description\": \"\"\n}\ n}\n]\n}","type":"dataframe","variable_name":"mm"}



at small n MAP trades a little bias for much lower variance; w → 0 as n grows so both converge.

Task 8 — Gradient-Based Optimisation of Softmax Regression (from scratch) (4 marks)

Model: $z_i = W^\top x_i$ (bias folded in), $p_i = \text{softmax}(z_i)$. Loss: $J(W) = -\frac{1}{N}\sum_i \sum_k Y_{ik} \log p_{ik} + \frac{\lambda}{2}\|W\|_2^2$.

Gradient derivation. For one sample, $\ell = -\sum_k y_k \log p_k$ with $p_k = e^{z_k} / \sum_j e^{z_j}$.

1. $\frac{\partial \log p_k}{\partial z_j} = \delta_{kj} - p_j$
2. $\frac{\partial \ell}{\partial z_j} = -\sum_k y_k (\delta_{kj} - p_j) = -y_j + p_j \sum_k y_k = p_j - y_j$ (since $\sum_k y_k = 1$)
3. $z_j = w_j^\top x \Rightarrow \frac{\partial \ell}{\partial w_j} = (p_j - y_j) x$
4. Batch form: $\nabla_W J = \frac{1}{N} X^\top (P - Y) + \lambda W$

Each update is driven by the error signal $P - Y$ (confident correct samples contribute ≈0; confident mistakes dominate) times the input x (large-variance feature directions get large updates → link to conditioning).

```
def softmax_rows(Zm): Zm = Zm - Zm.max(1, keepdims=True); Em =
np.exp(Zm); return Em / Em.sum(1, keepdims=True)
def add_bias(A): return np.hstack([A, np.ones((len(A), 1))])
def loss_grad(W, Xb, Yb, lam=1e-4):
    P = softmax_rows(Xb @ W)
    loss = -np.mean(np.sum(Yb * np.log(np.clip(P, 1e-12, 1)), 1)) +
lam / 2 * np.sum(W[:-1]**2)
    G = Xb.T @ (P - Yb) / len(Xb); G[:-1] += lam * W[:-1]
    return loss, G
def train_softmax(Xa, Ya, lr, batch, epochs, Xv=None, yv=None,
seed=0):
    r_ = np.random.default_rng(seed); W = np.zeros((Xa.shape[1],
Ya.shape[1])); hist, steps, t0 = [], [], time.time()
    for ep in range(epochs):
        idx = r_.permutation(len(Xa))
        for s0 in range(0, len(Xa), batch):
            b = idx[s0:s0 + batch]; l, G = loss_grad(W, Xa[b], Ya[b]);
W -= lr * G; steps.append(l)
        hist.append(loss_grad(W, Xa, Ya)[0])
        if not np.isfinite(hist[-1]) or hist[-1] > 1e6: hist +=
[np.nan] * (epochs - ep - 1); break
    return W, np.array(hist), np.array(steps), time.time() - t0

# numerical gradient check
Xg = add_bias(rng.standard_normal((20, 5))); Yg = np.eye(3)
[rng.integers(0, 3, 20)]; Wg = rng.standard_normal((6, 3)) * .1
_, Ga = loss_grad(Wg, Xg, Yg); Gn = np.zeros_like(Wg); h = 1e-6
for i in range(Wg.shape[0]):
    for j in range(Wg.shape[1]):
        Wp, Wm = Wg.copy(), Wg.copy(); Wp[i, j] += h; Wm[i, j] -= h
        Gn[i, j] = (loss_grad(Wp, Xg, Yg)[0] - loss_grad(Wm, Xg, Yg)
[0]) / (2 * h)
print('max |analytic - numerical gradient| =', f'{np.abs(Ga -
Gn).max():.2e}')
```

```
max |analytic - numerical gradient| = 2.37e-10
```

Conditioning of the Softmax-Regression Inputs

Hessian of the softmax cross-entropy

$$\nabla^2 J = \left(\operatorname{diag}(p) - p\,p^\top\right) \otimes \frac{X^\top X}{N}, \quad \lambda_{max}\left(\operatorname{diag}(p) - p\,p^\top\right) \le 1/2 \Rightarrow \lambda_{max}\left(\nabla^2 J\right) \le \frac{\lambda_{max}\left(X^\top X / N\right)}{2}$$

Critical learning rate (safe for every W)

$$\eta_{crit} = \frac{2}{\lambda_{max}(\nabla^2 J)} = \frac{4}{\lambda_{max}(X^\top X/N)}$$

Compared for unwhitened PCA-50 scores vs whitened (standardised) PCA-50 scores.

```
D8 = 50
pca50 = PCA(D8, random_state=SEED).fit(X_tr) F_raw = add_bias(pca50.transform(X_tr)); Fv_raw =
add_bias(pca50.transform(X_va))
sc50 = StandardScaler().fit(pca50.transform(X_tr))
F_w = add_bias(sc50.transform(pca50.transform(X_tr))); Fv_w =
add_bias(sc50.transform(pca50.transform(X_va)))
Ytr = np.eye(K)[y_tr]
def eig_info(F): ev = np.linalg.eigvalsh(F.T @ F / len(F)); return ev.max(), ev.max() / ev.min()
lmax_raw, kap_raw = eig_info(F_raw); lmax_w, kap_w = eig_info(F_w)
# Hessian = (diag(p) - pp^T)X^TX/N and λmax(diag(p) - pp^T) <= 1/2
-> λmax(H) <= λmax(X^TX/N)/2
eta_crit_raw, eta_crit_w = 4 / lmax_raw, 4 / lmax_w print(f'unwhitened PCA-50: λmax(X^TX/N) =
{f4(lmax_raw)}, κ =
{f4(kap_raw)}, η_crit = 4/λmax = {f4(eta_crit_raw)}') print(f'whitenedPCA-50: λmax(X^TX/N) = {f4(lmax_w)},
κ =
{f4(kap_w)}, η_crit = {f4(eta_crit_w)}')
```

```
unwhitened PCA-50: λmax(X^TX/N) = 16.7727, κ = 104.6735, η_crit = 4/λmax = 0.2385
whitenedPCA-50: λmax(X^TX/N) = 1.0011, κ = 1.0019, η_crit = 3.9957
```

Batch GD vs Mini-batch SGD vs SGD

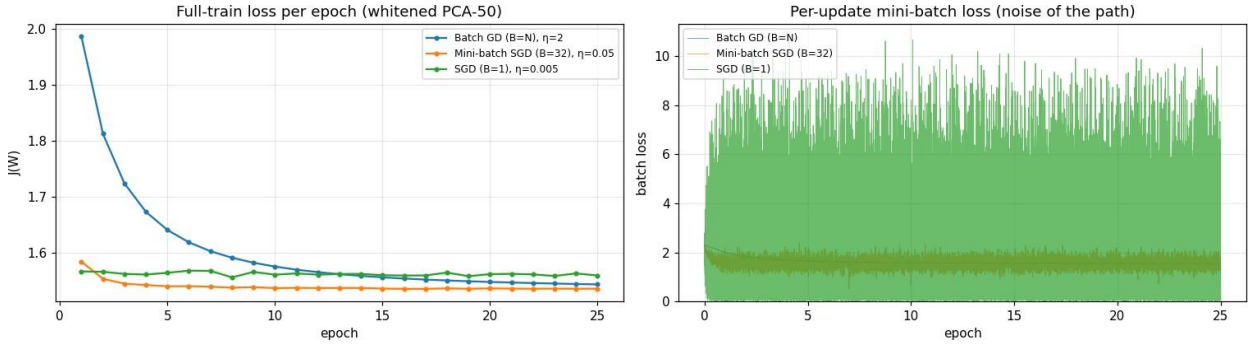
Update rule (batch B of size B)

$$W \leftarrow W - \eta \left[\frac{1}{B} X_B^\top (P_B - Y_B) + \lambda W \right]$$

$B=N$: batch GD (1 update/epoch) · $B=32$: mini-batch · $B=1$: SGD (N updates/epoch).

```
EP = 5 if TEST_MODE else 25
strategies = {'Batch GD (B=N)': (len(F_w), 0.5 * eta_crit_w), 'Mini-
batch SGD (B=32)': (32, 0.05), 'SGD (B=1)': (1, 0.005)}
fig, ax = plt.subplots(1, 2, figsize=(14, 4)); rows = []
for nm, (bs, lr) in strategies.items():
    W, hist, steps, tt = train_softmax(F_w, Ytr, lr, bs, EP)
    ups = int(np.ceil(len(F_w) / bs))
    ax[0].plot(np.arange(1, EP + 1), hist, 'o-', ms=3, label=f'{nm},
η={lr:.3g}')
    ax[1].plot(np.linspace(0, EP, len(steps)), steps, lw=.6, alpha=.7,
```

```
label=nm)
    rows.append([nm, bs, f4(lr), ups, f4(tt / EP), f4(hist[-1]),
f4(np.std(steps[-ups:])), f4(accuracy_score(y_va, (Fv_w @
W).argmax(1)))])
ax[0].set(title='Full-train loss per epoch (whitened PCA-50)',
xlabel='epoch', ylabel='J(W)'); ax[0].legend(fontsize=8)
ax[1].set(title='Per-update mini-batch loss (noise of the path)',
xlabel='epoch', ylabel='batch loss', ylim=(0, None));
ax[1].legend(fontsize=8)
plt.tight_layout(); plt.show()
display(pd.DataFrame(rows, columns=['strategy', 'batch', 'η',
'updates/epoch', 'sec/epoch', 'final loss', 'loss noise (sd, last
epoch)', 'val acc']))
```




```
{"summary":"{\n  \"name\": \"display(pd\",\n  \"rows\": 3,\n
\"fields\": [\n    {\n      \"column\": \"strategy\",\n
\"properties\": {\n        \"dtype\": \"string\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n          \"Batch
GD (B=N)\",\n          \"Mini-batch SGD (B=32)\",\n          \"SGD
(B=1)\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"batch\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 7329,\n        \"min\": 1,\n        \"max\": 12712,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          12712,\n
32,\n          1\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"\\
u03b7\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 1.1377757306839222,\n        \"min\": 0.005,\n
\"max\": 1.9978,\n        \"num_unique_values\": 3,\n
\"samples\": [\n          1.9978,\n          0.05,\n          0.005\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"updates/epoch\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
7226,\n        \"min\": 1,\n        \"max\": 12712,\n
\"num_unique_values\": 3,\n        \"samples\": [\n          1,\n
398,\n          12712\n        ],\n\"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"sec/epoch\",\n      \"properties\": {\n        \"dtype\":
```

\"number\",\n   \"std\": 0.4101374078687938,\n   \"min\": 0.0129,\n   \"max\": 0.7417,\n   \"num_unique_values\": 3,\n \"samples\": [\n0.0129,\n 0.0513,\n 0.7417\n   ],\n   \"semantic_type\": \"\",\n \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"final loss\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.012035087591427 58,\n   \"min\": 1.5365,\n   \"max\": 1.5601,\n   \"num_unique_values\": 3,\n \"samples\": [\n1.5442,\n 1.5365,\n 1.5601\n   ],\n   \"semantic_type\": \"\",\n \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"loss noise (sd, last epoch)\",\n   \"properties\": {\n \"dtype\": \"number\",\n   \"std\": 0.6531312604165669,\n \"min\": 0.0,\n   \"max\": 1.2156,\n \"num_unique_values\": 3,\n   \"samples\": [\n   0.0,\n 0.1937,\n   1.2156\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n \"column\": \"val acc\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 0.0012529964086141685,\n   \"min\": 0.4835,\n   \"max\": 0.486,\n   \"num_unique_values\": 3,\n \"samples\": [\n   0.4849,\n   0.486,\n   0.4835\n   ],\n   \"semantic_type\": \"\",\n \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

Effect of the Learning Rate

Batch GD with $\eta \in \{0.05, 0.5, 0.95, 3.0\} \times \eta_{crit}$ on ill-conditioned and well-conditioned inputs.

Stability condition

$$0 < \eta < \frac{2}{\lambda_{max}}$$

```
EPG = 30 if TEST_MODE else 150
mults = [0.05, 0.5, 0.95, 3.0]
fig, ax = plt.subplots(1, 2, figsize=(14, 4)); rows = []
for a, (nm, F, crit) in zip(ax, [('unwhitened (ill-conditioned)',
F_raw, eta_crit_raw), ('whitened (well-conditioned)', F_w,
eta_crit_w)]):
    for mlt in mults:
        lr = mlt * crit; _, hist, _, _ = train_softmax(F, Ytr, lr,
len(F), EPG)
        a.plot(hist, label=f'η = {mlt}·η_crit = {lr:.3g}')
        fin = hist[np.isfinite(hist)]
        rows.append([nm, mlt, f4(lr), f4(fin[-1]) if len(fin) else
np.nan, int(np.argmax(hist < 1.01 * np.nanmin(hist))) if len(fin) else
np.nan,
                     'diverged/oscillating' if (len(fin) < EPG or
np.any(np.diff(fin[:10]) > 0)) else 'stable'])
    a.set(title=f'Batch GD – {nm}', xlabel='epoch', ylabel='J(W)',
```

```
yscale='log'); a.legend(fontsize=8)
plt.tight_layout(); plt.show()
lr_tab = pd.DataFrame(rows, columns=['features', 'η / η_crit', 'η',
'final loss', 'epochs to within 1% of best', 'behaviour']);
display(lr_tab)
```

Points to analyse: batch GD – smooth path, one update per epoch (expensive per step, cheap per epoch in vectorised NumPy); SGD – N noisy updates per epoch, jagged path, reaches low loss in few epochs but plateaus at a noise floor; mini-batch – compromise, vectorised. Learning rate: small η → slow; η near η_crit → fastest; η > η_crit → overshoots/oscillates or diverges.
Unwhitened features have large κ (Task 4): the stable η is forced small by λ_{max} so low-variance PC directions learn very slowly → whitening (κ=1) converges far faster.

Task 9 — Experimental Performance across Dataset Scales (3 marks)

Scales use stratified subsets of the train+val pool; every model is evaluated on the same locked test set.

Metrics: $\text{Precision} = \frac{TP}{TP+FP}$, $\text{Recall} = \frac{TP}{TP+FN}$, $F_1 = \frac{2PR}{P+R}$ (macro-averaged over classes).

```
X_pool = np.vstack([X_tr, X_va]); y_pool = np.concatenate([y_tr,
y_va])
scales = [s for s in ([1000, 2000] if TEST_MODE else [1000, 5000,
10000]) if s < len(y_pool)] + [len(y_pool)]
def n_params(m):
    est = m.steps[-1][1] if hasattr(m, 'steps') else m
    extra = sum(s[1].components_.size for s in getattr(m, 'steps', [])
if isinstance(s[1], PCA))
    if isinstance(est, LogisticRegression): return est.coef_.size +
est.intercept_.size + extra
    if isinstance(est, KNeighborsClassifier): return
est.n_samples_fit_ * est.n_features_in_ + extra
    return sum(t.tree_.node_count for t in est.estimators_) + extra
# RF: number of tree nodes
def pipelines(feat):
    pre = [PCA(K_PCA, random_state=SEED)] if feat == 'PCA' else []
    return {'LogReg': make_pipeline(*pre, StandardScaler(),
LogisticRegression(max_iter=300)),
            'k-NN': make_pipeline(*pre, *([] if pre else
[StandardScaler()]), KNeighborsClassifier(5)),
            'RandomForest': make_pipeline(*pre,
RandomForestClassifier(100, n_jobs=-1, random_state=SEED))}
rows, fitted = [], {}
for n in scales:
    i = np.arange(len(y_pool)) if n == len(y_pool) else
train_test_split(np.arange(len(y_pool)), train_size=n,
stratify=y_pool, random_state=SEED)[0]
```

```
    for feat in ['raw', 'PCA']:
        for nm, m in pipelines(feat).items():
            t = time.time(); m.fit(X_pool[i], y_pool[i]); ttr =
time.time() - t
            t = time.time(); pr = m.predict(X_te); tin = (time.time()
- t) / len(X_te) * 1000
            P_, R_, F_, _ = precision_recall_fscore_support(y_te, pr,
average='macro', zero_division=0)
            rows.append([n, feat, nm, f4(accuracy_score(y_te, pr)),
f4(P_), f4(R_), f4(F_), f4(ttr), f4(tin), n_params(m),
f4(len(pickle.dumps(m)) / 2**20)])
            fitted[(n, feat, nm)] = (m, pr)
        print('done scale', n, feat)
perf = pd.DataFrame(rows, columns=['n', 'features', 'model',
'accuracy', 'precision', 'recall', 'F1', 'train s', 'infer ms/img',
'params', 'size MB'])
display(perf)
```

```
done scale 1000 raw
done scale 1000 PCA
done scale 5000 raw
done scale 5000 PCA
done scale 10000 raw
done scale 10000 PCA
done scale 15436 raw
done scale 15436 PCA
```


```
{"summary":"{\n  \"name\": \"perf\",\n  \"rows\": 24,\n  \"fields\":
[\n    {\n      \"column\": \"n\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 5529,\n        \"min\":
1000,\n        \"max\": 15436,\n\"num_unique_values\": 4,\n
\"samples\": [\n          5000,\n          15436,\n          1000\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"features\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 2,\n        \"samples\": [\n          \"PCA\",\
n          \"raw\"\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"model\",\n      \"properties\": {\n        \"dtype\": \"category\",\
n        \"num_unique_values\": 3,\n        \"samples\": [\n
\"LogReg\",\n          \"k-NN\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"accuracy\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
0.08395075409023339,\n        \"min\": 0.3609,\n        \"max\":
0.6575,\n        \"num_unique_values\": 24,\n        \"samples\": [\n
0.6101,\n          0.4515\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"precision\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.09997729080914443,\n
```

```
\"min\": 0.2665,\n        \"max\": 0.6185,\n
\"num_unique_values\": 24,\n        \"samples\": [\n          0.5711,\
n          0.4436\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"recall\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.0770868030163099,\n        \"min\": 0.186,\n
\"max\": 0.5143,\n        \"num_unique_values\": 24,\n
\"samples\": [\n          0.4594,\n          0.3835\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"F1\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.08125228915392861,\n
\"min\": 0.1777,\n        \"max\": 0.5358,\n
\"num_unique_values\": 24,\n        \"samples\": [\n          0.4799,\
n          0.3652\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"train s\",\n      \"properties\": {\n        \"dtype\": \"number\",\
n        \"std\": 27.9030407509099,\n        \"min\": 0.0601,\n
\"max\": 103.2821,\n        \"num_unique_values\": 24,\n
\"samples\": [\n          18.0047,\n          6.4966\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"infer ms/img\",\n
\"properties\": {\n\"dtype\": \"number\",\n            \"std\":
0.6988845316811557,\n        \"min\": 0.0255,\n        \"max\":
3.0652,\n\"num_unique_values\": 24,\n \"samples\":        [\n
0.0289,\n        0.31\n          ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"params\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 14854469,\n        \"min\": 40970,\n
\"max\": 63225856,\n        \"num_unique_values\": 18,\n
\"samples\": [\n          40970,\n          4096000\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"size MB\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
58.083835065278144,\n        \"min\": 0.4075,\n\"max\":
241.4002,\n\"num_unique_values\": 18,\n    \"samples\":          [\
n          0.4075,\n          15.7275\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n ]\n}","type":"dataframe","variable_name":"perf"}
```

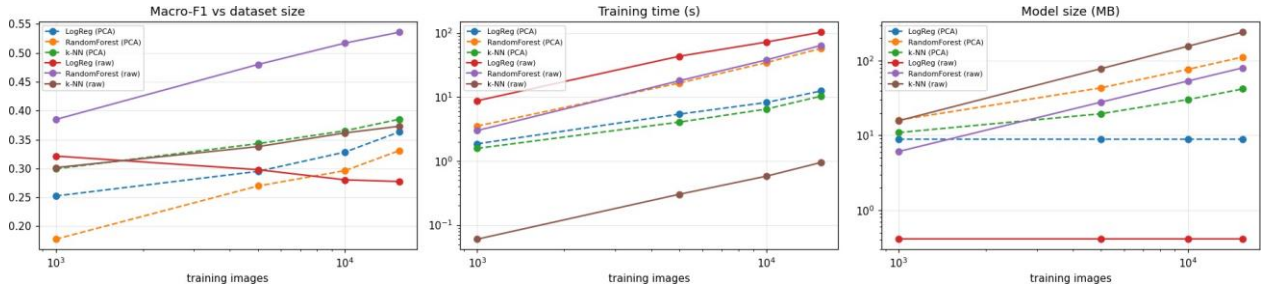

Performance Plots and Confusion Matrix

Confusion matrix M_{ij} = number of test images of true class i predicted as class j; off-diagonal maxima = most-confused pairs. Per-class metrics

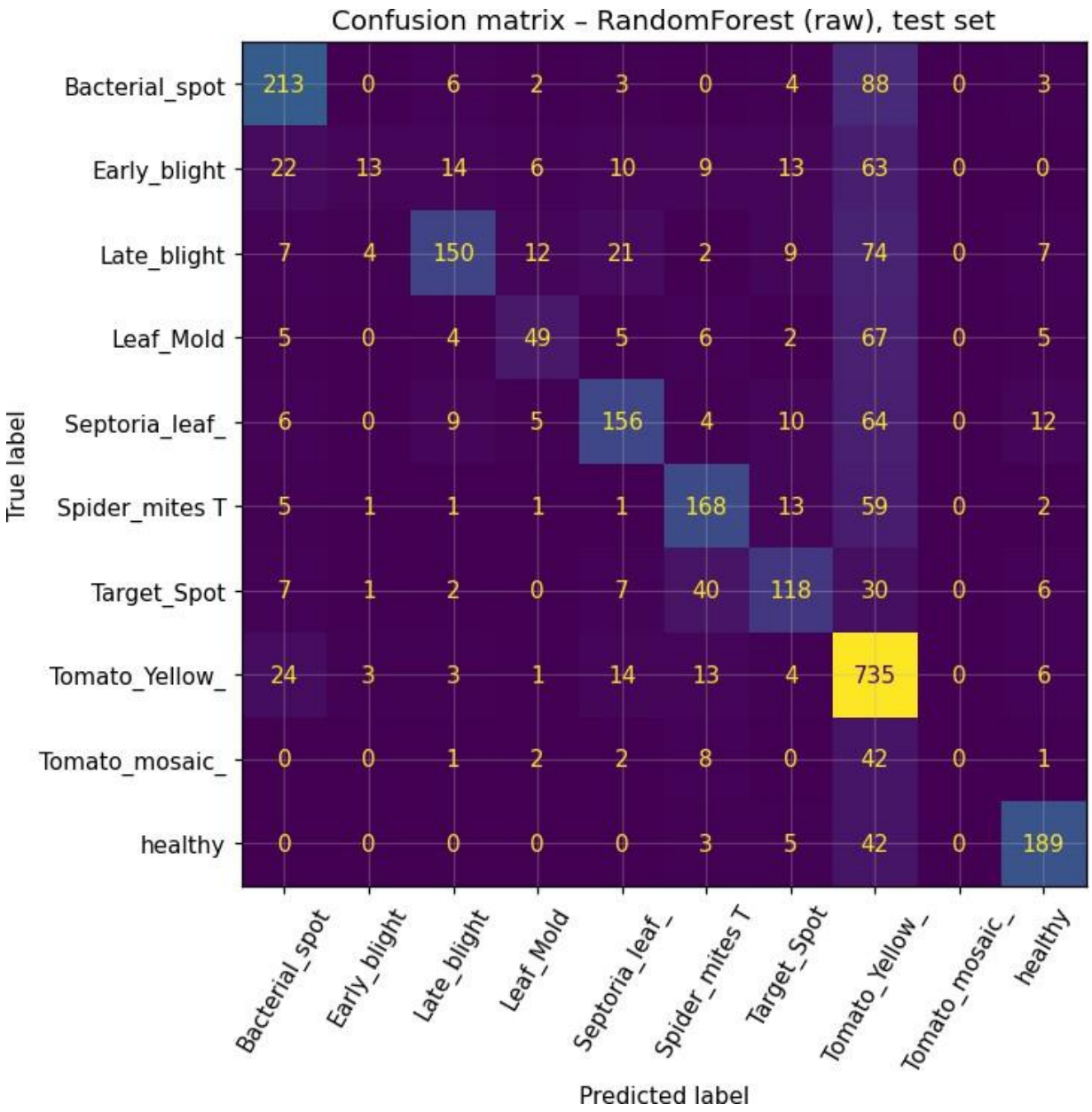
$$\text{Precision}_k = \frac{M_{kk}}{\sum_i M_{ik}}, \text{Recall}_k = \frac{M_{kk}}{\sum_j M_{kj}}$$

```
fig, ax = plt.subplots(1, 3, figsize=(17, 4))
for (feat, nm), g in perf.groupby(['features', 'model']):
    ls = '-' if feat == 'raw' else '--'
    ax[0].plot(g.n, g.F1, 'o' + ls, label=f'{nm} ({feat})');
ax[1].plot(g.n, g['train s'], 'o' + ls, label=f'{nm} ({feat})')
    ax[2].plot(g.n, g['size MB'], 'o' + ls, label=f'{nm} ({feat})')
for a, t in zip(ax, ['Macro-F1 vs dataset size', 'Training time (s)',
'Model size (MB)']):
    a.set(title=t, xlabel='training images', xscale='log');
a.legend(fontsize=7)
ax[1].set_yscale('log'); ax[2].set_yscale('log'); plt.tight_layout();
plt.show()

best_row = perf[perf.n == perf.n.max()].sort_values('F1').iloc[-1]
m_best, pr_best = fitted[(best_row.n, best_row.features,
best_row.model)]
print('Best at full scale:', best_row.model, best_row.features, '| F1
=', best_row.F1)
cm = confusion_matrix(y_te, pr_best)
fig, a = plt.subplots(figsize=(8, 7)); ConfusionMatrixDisplay(cm,
display_labels=[c[:14] for c in CLASSES]).plot(ax=a,
xticks_rotation=60, colorbar=False)
a.set_title(f'Confusion matrix — {best_row.model}
({best_row.features}), test set'); plt.tight_layout(); plt.show()
off = cm.copy().astype(float); np.fill_diagonal(off, 0)
pairs = [(CLASSES[i], CLASSES[j], int(off[i, j])) for i, j in
zip(*np.unravel_index(np.argsort(off, None)[::-1][:5], off.shape))]
display(pd.DataFrame(pairs, columns=['true class', 'predicted as',
'count']))
P_c, R_c, F_c, S_c = precision_recall_fscore_support(y_te, pr_best,
zero_division=0)
display(pd.DataFrame({'precision': P_c, 'recall': R_c, 'F1': F_c,
'support': S_c}, index=CLASSES))
```




```
Best at full scale: RandomForest raw | F1 = 0.5358
```




```
{"summary":"{\n \"name\": \"display(pd\",\n \"rows\": 5,\n \"fields\": [\n {\n \"column\": \"true class\",\n \"properties\": {\n \"dtype\": \"string\",\n \"num_unique_values\": 5,\n \"samples\": [\n \"Late_blight\",\n \"Early_blight\",\n \"Leaf_Mold\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"predicted as\",\n \"properties\": {\n \"dtype\": \"category\",\n\"num_unique_values\": 1,\n\"samples\": [\n\"Tomato_Yellow_Leaf_Curl_Virus\"\n],\n \"semantic_type\": \"\",\n\"description\": \"\"\n}\
```

```
n    },\n{\n \"column\": \"count\",\n    \"properties\":          {\
n        \"dtype\": \"number\",\n        \"std\": 10,\n
\"min\": 63,\n        \"max\": 88,\n        \"num_unique_values\": 5,\
n        \"samples\": [\n          74\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n ]\n}","type":"dataframe"}

{"summary":"{\n  \"name\": \"display(pd\",\n  \"rows\": 10,\n
\"fields\": [\n    {\n      \"column\": \"precision\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.23112542482964327,\n        \"min\": 0.0,\n        \"max\":
0.8181818181818182,\n        \"num_unique_values\": 10,\n
\"samples\": [\n          0.0,\n          0.5909090909090909,\n
0.6640316205533597\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"recall\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.29230861098736416,\n        \"min\": 0.0,\n        \"max\":
0.9153175591531756,\n        \"num_unique_values\": 10,\n
\"samples\": [\n          0.0,\n          0.08666666666666667,\n
0.6693227091633466\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"F1\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.2616589793067083,\n        \"min\": 0.0,\n        \"max\":
0.8042553191489362,\n        \"num_unique_values\": 10,\n
\"samples\": [\n          0.0,\n          0.1511627906976744,\n
0.6666666666666666\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"support\",\n      \"properties\": {\n        \"dtype\": \"number\",\
n        \"std\": 202,\n        \"min\": 56,\n        \"max\": 803,\n
\"num_unique_values\": 10,\n        \"samples\": [\n          56,\n
150,\n          251\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


Points to analyse: which models keep improving with n (k-NN, RF – non-parametric/high capacity) vs saturate (LR – fixed capacity); costs: k-NN has ~0 training cost but inference and size grow with n; LR size is constant; RF size grows with n. Confused pairs are usually diseases that differ mainly in colour or fine texture (lost by 64×64 grayscale, Task 1).

Task 10 — Theory vs Experiment

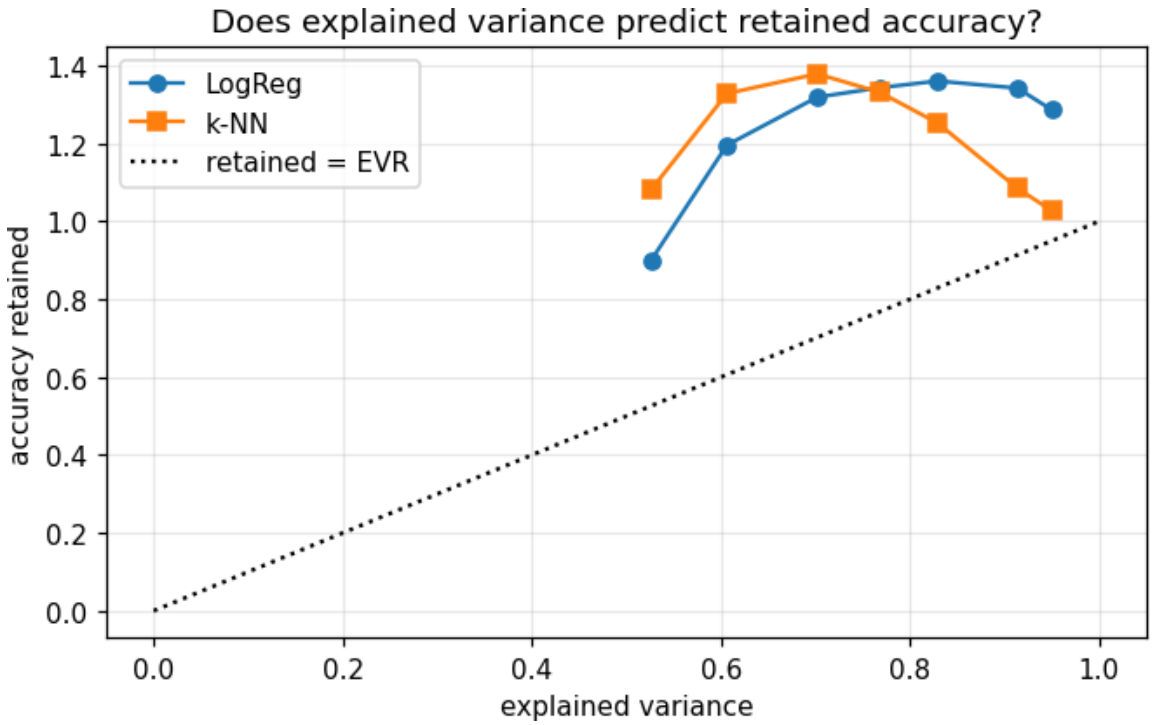
```
mk10 = {'LogReg': lambda: make_pipeline(StandardScaler(),
LogisticRegression(max_iter=300)), 'k-NN': lambda:
KNeighborsClassifier(5)}
acc_raw = {'LogReg': accuracy_score(y_va, mk10['LogReg']().fit(X_tr,
y_tr).predict(X_va)),
           'k-NN': accuracy_score(y_va,
make_pipeline(StandardScaler(), KNeighborsClassifier(5)).fit(X_tr,
y_tr).predict(X_va))}
rows = []
```

```
for k in sorted(set([5, 10, 25, 50, 100, K_PCA, min(300, len(S) -
1)])):
    if k <= K_PCA: Zt, Zv = Z_tr[:, :k], Z_va[:, :k]           #
sklearn PCA components are ordered, so slicing = PCA-k
    else: p_ = PCA(k, random_state=SEED).fit(X_tr); Zt, Zv =
p_.transform(X_tr), p_.transform(X_va)
    rows.append([k, f4(cum[k - 1])] + [f4(accuracy_score(y_va,
mk().fit(Zt, y_tr).predict(Zv)) / acc_raw[nm]) for nm, mk in
mk10.items()])
t10 = pd.DataFrame(rows, columns=['k', 'explained variance', 'LogReg
acc retained (PCA/raw)', 'k-NN acc retained (PCA/raw)']); display(t10)
plt.figure(figsize=(7, 4)); plt.plot(t10['explained variance'],
t10.iloc[:, 2], 'o-', label='LogReg'); plt.plot(t10['explained
variance'], t10.iloc[:, 3], 's-', label='k-NN')
plt.plot([0, 1], [0, 1], 'k:', label='retained = EVR');
plt.xlabel('explained variance'); plt.ylabel('accuracy retained');
plt.legend(); plt.title('Does explained variance predict retained
accuracy?'); plt.show()

print('Optimisation: κ unwhitened =', f4(kap_raw), '-> predicted
contraction', f4((kap_raw - 1) / (kap_raw + 1)),
      '| κ whitened =', f4(kap_w), '-> predicted', f4((kap_w - 1) /
(kap_w + 1)))
display(lr_tab[lr_tab['η / η_crit'] == 0.5])
print('Bias-variance (Task 7) vs capacity curve (Task 6):');
display(bvt[['bias²', 'variance', '0-1 error']])
```


```
{"summary":"{\n  \"name\": \"t10\",\n  \"rows\": 7,\n  \"fields\": [\n
{\n      \"column\": \"k\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 207,\n        \"min\": 5,\n
\"max\": 560,\n        \"num_unique_values\": 7,\n        \"samples\":
[\n          5,\n          10,\n          300\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"explained variance\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.1560443930378261,\n        \"min\": 0.526199996471405,\n
\"max\": 0.949999988079071,\n        \"num_unique_values\": 7,\n
\"samples\": [\n          0.526199996471405,\n
0.6065000295639038,\n          0.9136999845504761\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"LogReg acc retained (PCA/raw)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.16383915312048608,\n        \"min\": 0.9,\n        \"max\": 1.3606,\
n        \"num_unique_values\": 7,\n        \"samples\": [\n
0.9,\n          1.196,\n          1.3424\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"k-NN acc retained (PCA/raw)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.14406787091010761,\n        \"min\": 1.0276,\n        \"max\":
1.3789,\n        \"num_unique_values\": 7,\n        \"samples\": [\n
```

1.0813,\n 1.3285,\n 1.0854\n],\n \"semantic_type\": \"\",\n\"description\": \"\"\n}\ n}\n]\n}","type":"dataframe","variable_name":"t10"}



Optimisation: κ unwhitened = 104.6735 -> predicted contraction 0.9811 | κ whitened = 1.0019 -> predicted 0.001

{"summary":"{\n \"name\": \"print('Bias-variance (Task 7) vs capacity curve (Task 6):'); display(bvt[['bias\\u00b2', 'variance', '0-1 error']])\",\n \"rows\": 2,\n \"fields\": [\n{\n \"column\": \"features\",\n\"properties\": {\n \"dtype\": \"string\",\n [\n \"num_unique_values\": 2,\n \"samples\": \"whitened (well-conditioned)\",\n \"unwhitened (ill-conditioned)\"\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"\\u03b7 / \\u03b7_crit\",\n\"properties\": {\n\"dtype\": \"number\",\n \"std\": 0.0,\n \"min\": 0.5,\n \"max\": 0.5,\n \"num_unique_values\": 1,\n \"samples\": [\n0.5\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n },\n {\n \"column\": \"\\u03b7\",\n\"properties\": {\n\"dtype\": \"number\",\n \"std\": 1.3283707991370481,\n\"min\": 0.1192,\n \"max\": 1.9978,\n \"num_unique_values\": 2,\n \"samples\": [\n 1.9978\n],\n \"semantic_type\": \"\",\n \"description\": \"\"\n }\n},\n{\n\"column\": \"final loss\",\n

```
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.040941482630700986,\n        \"min\": 1.5335,\n        \"max\":
1.5914,\n        \"num_unique_values\": 2,\n        \"samples\": [\n
1.5335\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"epochs to within 1% of best\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 69,\n        \"min\": 19,\n
\"max\": 117,\n        \"num_unique_values\": 2,\n        \"samples\":
[\n          19\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"behaviour\",\n      \"properties\": {\n        \"dtype\":
\"string\",\n\"num_unique_values\": 1,\n     \"samples\":
[\n          \"stable\"\n        ],\n        \"semantic_type\": \"\",\
n        \"description\": \"\"\n      }\n    }\n ]\
n}","type":"dataframe"}
```


Bias-variance (Task 7) vs capacity curve (Task 6):


```
{"summary":"{\n  \"name\": \"print('Bias-variance (Task 7) vs capacity
curve (Task 6):'); display(bvt[['bias\\u00b2', 'variance', '0-1
error']])\",\n  \"rows\": 2,\n  \"fields\": [\n    {\n
\"column\": \"bias\\u00b2\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.06703372285648478,\n
\"min\": 0.7001,\n        \"max\": 0.7949,\n
\"num_unique_values\": 2,\n        \"samples\": [\n          0.7001,\n
0.7949\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"variance\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.47560002102607185,\n        \"min\":
0.0111,\n        \"max\": 0.6837,\n        \"num_unique_values\": 2,\n
\"samples\": [\n          0.6837,\n          0.0111\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"0-1 error\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.009475230867899713,\n        \"min\": 0.6785,\n        \"max\":
0.6919,\n        \"num_unique_values\": 2,\n        \"samples\": [\n
0.6919,\n          0.6785\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    }\n ]\
n}","type":"dataframe"}
```


Points to analyse: (1) accuracy retained usually exceeds EVR at small k – PCA keeps the high-variance (discriminative) directions and drops pixel noise; the curve is *not* the identity because variance ≠ class information. (2) Whitening should converge faster, as κ predicts; the GD bound is only approximate because softmax-CE is not quadratic (curvature shrinks as p becomes confident). (3) Depth-2 tree = high bias ↔ left side of Task 6 curves; depth-∞ = high variance ↔ large generalisation gap. Explain any disagreement (finite samples, non-quadratic loss, 0-1 vs Brier loss).

Task 11 — Is Deep Learning Justified? (4 marks)

Evidence measured below: (a) curse of dimensionality – distance concentration $\frac{d_{max} - d_{min}}{d_{min}}$;

(b) local constancy – does a tiny, label-preserving change (shifting the leaf by a few pixels) keep the image close in pixel space?;

(c) manifold structure – intrinsic dimension by the TwoNN estimator $\hat{d} = n / \sum_i \log\left(r^{(i)}_2 / r^{(i)}_1\right)$.

```
def concentration(A, Bm):
    D = euclidean_distances(A, Bm); return f4(np.mean((D.max(1) -
D.min(1)) / D.min(1)))
nq = 200
print('distance contrast (dmax-dmin)/dmin - raw 4096 :',
concentration(X_va[:nq], X_tr[:2000]))
print(f'distance contrast - PCA-{K_PCA}            :',
concentration(Z_va[:nq], Z_tr[:2000]))

# Local constancy: distance to a 3-pixel shifted copy vs distance to
the nearest image of ANOTHER class
idx_q = rng.permutation(len(X_img))[:nq]
shift = np.roll(X_img[idx_q], 3, axis=2).reshape(nq, -1)
orig = X_img[idx_q].reshape(nq, -1)
d_shift = np.linalg.norm(orig - shift, axis=1)
D_all = euclidean_distances(orig, X_tr[:3000])
other = y_tr[:3000][None, :] != y[idx_q][:, None]
d_other = np.where(other, D_all, np.inf).min(1)
print('fraction where the 3-px shifted SAME leaf is farther than the
nearest DIFFERENT-class image:', f4((d_shift > d_other).mean()))

def twonn(A):
    D = euclidean_distances(A); np.fill_diagonal(D, np.inf); D.sort(1)
    mu = D[:, 1] / np.maximum(D[:, 0], 1e-12); mu = mu[mu > 1]; return
f4(len(mu) / np.log(mu).sum())
print('TwoNN intrinsic dimension (1500 images):', twonn(X_tr[:1500]),
'| ambient = 4096 | k95 =', K_PCA)
```

```
distance contrast (dmax-dmin)/dmin - raw 4096 : 3.4196999073028564
distance contrast - PCA-560            : 3.7441000938415527
fraction where the 3-px shifted SAME leaf is farther than the nearest
DIFFERENT-class image: 0.335
TwoNN intrinsic dimension (1500 images): 29.1299991607666 | ambient =
4096 | k95 = 560
```

Reference: Small CNN (learned hierarchical features)

Convolution (shared local filter, translation-equivariant)

$$h_{ij} = \sigma\left(\sum_{u,v} w_{uv}\, x_{i+u,\, j+v} + b \right)$$

3 conv blocks → global average pooling → softmax. Used only as evidence for the analysis; runs in Colab with TensorFlow.

```
RUN_CNN = not TEST_MODE      # small CNN as a reference point for the
analysis (uses TensorFlow in Colab)
cnn_res = None
if RUN_CNN:
    try:
        import tensorflow as tf
        tf.random.set_seed(SEED)
        cnn = tf.keras.Sequential([tf.keras.layers.Input((IMG, IMG,
1)),
            tf.keras.layers.Conv2D(16, 3, activation='relu',
padding='same'), tf.keras.layers.MaxPool2D(),
            tf.keras.layers.Conv2D(32, 3, activation='relu',
padding='same'), tf.keras.layers.MaxPool2D(),
            tf.keras.layers.Conv2D(64, 3, activation='relu',
padding='same'), tf.keras.layers.GlobalAveragePooling2D(),
            tf.keras.layers.Dense(K, activation='softmax')])
        cnn.compile('adam', 'sparse_categorical_crossentropy',
metrics=['accuracy'])
        t = time.time(); cnn.fit(X_tr.reshape(-1, IMG, IMG, 1), y_tr,
validation_data=(X_va.reshape(-1, IMG, IMG, 1), y_va), epochs=10,
batch_size=64, verbose=2); ttr = time.time() - t
        pr = cnn.predict(X_te.reshape(-1, IMG, IMG, 1),
verbose=0).argmax(1)
        one = X_te[:1].reshape(1, IMG, IMG, 1); cnn(one); t =
time.time(); [cnn(one) for _ in range(50)]; tin = (time.time() - t) /
50 * 1000
        cnn_res = dict(accuracy=f4(accuracy_score(y_te, pr)),
F1=f4(precision_recall_fscore_support(y_te, pr, average='macro')[2]),
                       params=cnn.count_params(),
size_MB=f4(cnn.count_params() * 4 / 2**20), train_s=f4(ttr),
infer_ms_single=f4(tin))
        print(cnn_res)
    except ImportError:
        print('TensorFlow not available - skipping CNN reference')
```

```
Epoch 1/10
199/199 - 13s - 63ms/step - accuracy: 0.3008 - loss: 2.0748 -
val_accuracy: 0.3102 - val_loss: 1.9966
Epoch 2/10
199/199 - 1s - 7ms/step - accuracy: 0.3245 - loss: 1.9297 -
val_accuracy: 0.3333 - val_loss: 1.8685
Epoch 3/10
199/199 - 1s - 7ms/step - accuracy: 0.3643 - loss: 1.8175 -
val_accuracy: 0.3935 - val_loss: 1.7395
```

```
Epoch 4/10
199/199 - 1s - 7ms/step - accuracy: 0.4230 - loss: 1.6711 -
val_accuracy: 0.4413 - val_loss: 1.6179
Epoch 5/10
199/199 - 1s - 7ms/step - accuracy: 0.4473 - loss: 1.5711 -
val_accuracy: 0.4670 - val_loss: 1.5298
Epoch 6/10
199/199 - 1s - 7ms/step - accuracy: 0.4678 - loss: 1.4956 -
val_accuracy: 0.4901 - val_loss: 1.4561
Epoch 7/10
199/199 - 1s - 7ms/step - accuracy: 0.4875 - loss: 1.4352 -
val_accuracy: 0.5062 - val_loss: 1.4131
Epoch 8/10
199/199 - 1s - 7ms/step - accuracy: 0.5012 - loss: 1.3956 -
val_accuracy: 0.5143 - val_loss: 1.3835
Epoch 9/10
199/199 - 1s - 7ms/step - accuracy: 0.5092 - loss: 1.3666 -
val_accuracy: 0.5202 - val_loss: 1.3600
Epoch 10/10
199/199 - 1s - 7ms/step - accuracy: 0.5156 - loss: 1.3430 -
val_accuracy: 0.5264 - val_loss: 1.3385
{'accuracy': 0.5206, 'F1': 0.3575, 'params': 23946, 'size_MB': 0.0913,
'train_s': 25.5279, 'infer_ms_single': 7.5206}
```

Summary Table: Best Classical Models vs CNN

Full-scale test results with parameter count, size and inference time side by side.

```
summary = perf[perf.n == perf.n.max()].sort_values('F1', ascending=False)[['features', 'model',
'accuracy', 'F1', 'params', 'size MB', 'infer ms/img']]
if cnn_res: summary.loc[len(summary)] = ['image', 'small CNN (reference)', cnn_res['accuracy'],
cnn_res['F1'], cnn_res['params'], cnn_res['size_MB'], cnn_res['infer_ms_single']]
display(summary)
```


```
{"summary":"{\n \"name\": \"summary\",\n \"rows\": 7,\n \"fields\": [\n{\n\"column\":
\"features\",\n\"properties\": {\n


\"dtype\": \"category\",\n                          \"num_unique_values\": 3,\n
\"samples\": [\n                    \"raw\",\n                    \"PCA\",\n
\"image\"\n],\n\"semantic_type\": \"\",\n
\"description\": \"\"\n}\n},\n{\n\"column\":
\"model\",\n\"properties\": {\n\"dtype\": \"string\",\n
\"num_unique_values\": 4,\n\"samples\": [\n\"k-NN\",\n\"small CNN (reference)\",\n

\"RandomForest\"\n                    ],\n                \"semantic  type\": \"\",\n
\"description\": \"\"\n                    }\n            },\n        {\n            \"column\":
\"accuracy\",\n                \"properties\": {\n                    \"dtype\":
\"number\".\n                \"std\": 0.08851355290360367,\n                    \"min\":
```

```
0.3645,\n        \"max\": 0.6575,\n        \"num_unique_values\": 7,\n
\"samples\": [\n0.6575,\n 0.4648,\n
0.3645\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"F1\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.07943534058171675,\n        \"min\": 0.2774,\n
\"max\": 0.5358,\n        \"num_unique_values\": 7,\n
\"samples\": [\n0.5358,\n 0.3853,\n
0.2774\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"params\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 23145082,\n        \"min\": 23946,\n        \"max\":
63225856,\n        \"num_unique_values\": 7,\n        \"samples\": [\n
583634,\n          10937920,\n          40970\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"size MB\",\n      \"properties\":
{\n        \"dtype\": \"number\",\n        \"std\":
87.09198398277644,\n\"min\": 0.0913,\n  \"max\":
241.4002,\n\"num_unique_values\": 7,\n  \"samples\":    [\n
80.1887,\n          41.8661,\n          0.4075\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"infer ms/img\",\n
\"properties\": {\n\"dtype\": \"number\",\n                \"std\":
2.823963974291322,\n        \"min\": 0.0408,\n        \"max\":
7.5206,\n        \"num_unique_values\": 7,\n        \"samples\": [\n
0.0408,\n          0.5193,\n0.0581\n                ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    }\n ]\n}","type":"dataframe","variable_name":"summary"}
```


Points to analyse (trace each to a number above):

- Remaining limitations of the best tuned baseline: residual confusions (Task 9), high variance/size of k-NN/RF (Task 7, 9).
- Curse of dimensionality: low distance contrast in 4096-D → "nearest" neighbours are barely nearer than the rest.
- Local constancy fails in pixel space: a 3-pixel shift (same leaf, same disease) is often *farther* than an image of a different disease → smoothness prior of k-NN/kernels is placed in the wrong space.
- Manifold: intrinsic dimension $\ll 4096$ → data lies on a low-dimensional, *non-linear* manifold; PCA (linear) needs k95 dims to approximate it.
- Need for learned hierarchical features: edges → lesion spots → leaf patterns, translation-invariant (convolution + pooling) – exactly what (a)–(c) show is missing.
- Trade-offs: CNN needs more data/compute and is less interpretable than LR weights/PCs.

Task 12 — Day-4 Design Challenge: 38 classes, 20 % noisy labels, 50 ms / 5 MB

```
X38_tr, X38_te, y38_tr, y38_te = train_test_split(X38, y38,
    test_size=0.3, stratify=y38, random_state=SEED)
    c38 = np.unique(y38); K38 = len(c38); m38 = {c: i for i, c in
    enumerate(c38)}
y38_tr = np.array([m38[c] for c in y38_tr]); y38_te = np.array([m38[c]
    for c in y38_te])
    def add_noise(lbl, rate, k, seed=SEED):
        r_ = np.random.default_rng(seed); out = lbl.copy(); flip =
    r_.random(len(lbl)) < rate
        out[flip] = (lbl[flip] + r_.integers(1, k, flip.sum())) % k    #
    uniform over the OTHER classes
     return out
    y38_noisy = add_noise(y38_tr, 0.20, K38)
    p38 = np.bincount(y38_tr, minlength=K38) / len(y38_tr)
    A = pd.DataFrame({'K': [K, K38], 'H(p) bits': [f4(H(p_cls)),
    f4(H(p38))], 'log2 K (max)': [f4(np.log2(K)), f4(np.log2(K38))],
                      'CE at init = log2 K': [f4(np.log2(K)),
    f4(np.log2(K38))], 'chance acc 1/K': [f4(1 / K), f4(1 / K38)],
                      'majority acc': [f4(p_cls.max()), f4(p38.max())]},
    index=['original tomato', '38-class']); display(A)
print('actual flipped fraction =', f4((y38_noisy != y38_tr).mean())) eta
    = 0.2; print(f'With symmetric noise η=0.2 a perfect classifier scores
    only {f4(1 - eta)} accuracy on NOISY labels; '
                 f'noise adds ≈ H_b(0.2) + 0.2·log2(K-1) = {f4(-
    (0.2*np.log2(0.2) + 0.8*np.log2(0.8)) + 0.2*np.log2(K38-1))} bits of
    irreducible label entropy')
```


```
    {"summary":"{\n  \"name\": \"A\",\n  \"rows\": 2,\n  \"fields\": [\n
    {\n      \"column\": \"K\",\n      \"properties\": {\n
    \"dtype\": \"number\",\n        \"std\": 19,\n        \"min\": 10,\n
\"max\": 38,\n   \"num_unique_values\": 2,\n            \"samples\": [\n
                38,\n          10\n        ],\n        \"semantic_type\":
    \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
    \"column\": \"H(p) bits\",\n      \"properties\": {\n
    \"dtype\": \"number\",\n        \"std\": 1.5682921299936436,\n
    \"min\": 3.03,\n        \"max\": 5.2479,\n
\"num_unique_values\": 2,\n   \"samples\": [\n                    5.2479,\n
    3.03\n        ],\n        \"semantic_type\": \"\",\n
    \"description\": \"\"\n      }\n    },\n    {\n      \"column\":
    \"log2 K (max)\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n            \"std\": 1.3618876605652903,\n        \"min\":
    3.3219,\n        \"max\": 5.2479,\n        \"num_unique_values\": 2,\n
    \"samples\": [\n          5.2479,\n          3.3219\n        ],\n
\"semantic_type\": \"\",\n            \"description\": \"\"\n      }\
    n    },\n    {\n      \"column\": \"CE at init = log2 K\",\n
    \"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
```

```
1.3618876605652903,\n                    \"min\": 3.3219,\n                    \"max\":
5.2479,\n                \"num_unique_values\": 2,\n                    \"samples\": [\n
5.2479,\n                    3.3219\n                ],\n                \"semantic_type\":
\"\",\n\"description\": \"\"\n}\n},\n{\n
\"column\": \"chance acc 1/K\".\n\"properties\": {\n
\"dtype\": \"number\",\n                    \"std\": 0.05211376977344855,\n
\"min\": 0.0263,\n                    \"max\": 0.1,\n
\"num_unique_values\": 2,\n\"samples\": [\n0.0263,\n 0.1\n],\n\"semantic_type\": \"\",\n
\"description\": \"\"\n}\n},\n{\n
                                                    \"column\":
\"majority acc\",\n                    \"properties\": {\n                    \"dtype\":
\"number\".\n                \"std\": 0.1899995921048253,\n                    \"min\":
0.0263,\n\"max\": 0.295,\n\"num_unique_values\": 2,\n
\"samples\": [\n0.0263,\n0.295\n                                                    ],\n
\"semantic_type\": \"\",\n\"description\": \"\"\n                                    }\
n}\n ]\n}","type":"dataframe","variable_name":"A"}
```


actual flipped fraction = 0.2058
With symmetric noise η=0.2 a perfect classifier scores only 0.8 accuracy on NOISY labels; noise adds ≈ H_b(0.2) + 0.2·log2(K-1) = 1.7638 bits of irreducible label entropy

A. Impact (points to analyse): label space 10 → 38: entropy ceiling and initial CE rise to $\log_2 38 = 5.2479$ bits, chance accuracy falls, more boundaries for a linear model, fewer images per class. 20 % noise: irreducible error ↑ (Bayes error on noisy labels ≥ 0.2), *variance* ↑ for high-capacity models that memorise flipped labels, measured validation error is biased upwards if validation labels are noisy. Broken assumptions: clean labels, unlimited memory/latency (k-NN stores the whole training set), 10 classes.

B. Measure Each Candidate under the New Conditions

Every model is trained once on clean and once on noisy labels and tested on clean labels. Single-image latency is measured in Colab and scaled to a low-cost phone:

$$t_{phone} \approx t_{Colab} \times \text{PHONE_SLOWDOWN}$$

```
PHONE_SLOWDOWN = 8   # ponytail: assumed low-cost-phone CPU ≈ 8×
slower than a Colab CPU core; replace with an on-device measurement
p38 = PCA(K_PCA, random_state=SEED).fit(X38_tr)
def latency_ms(m, Xq, reps=100):
    m.predict(Xq[:1]); t = time.time(); [m.predict(Xq[i:i + 1]) for i
in range(reps)]; return (time.time() - t) / reps * 1000
cands = {'LogReg + PCA': lambda: make_pipeline(PCA(K_PCA,
random_state=SEED), StandardScaler(),
LogisticRegression(max_iter=300)),
         'k-NN k=1 + PCA': lambda: make_pipeline(PCA(K_PCA,
random_state=SEED), KNeighborsClassifier(1)),
         'k-NN k=15 + PCA': lambda: make_pipeline(PCA(K_PCA,
random_state=SEED), KNeighborsClassifier(15)),
```

```
        'RandomForest + PCA': lambda: make_pipeline(PCA(K_PCA,
random_state=SEED), RandomForestClassifier(100, n_jobs=1,
random_state=SEED))}
rows = []
for nm, mk in cands.items():
    accs = []
    for lbl in (y38_tr, y38_noisy):
        m = mk().fit(X38_tr, lbl); accs.append(accuracy_score(y38_te,
m.predict(X38_te)))
    lat = latency_ms(m, X38_te); size = len(pickle.dumps(m)) / 2**20
    rows.append([nm, f4(accs[0]), f4(accs[1]), f4(accs[0] - accs[1]),
f4(lat), f4(lat * PHONE_SLOWDOWN), f4(size),
                 'PASS' if lat * PHONE_SLOWDOWN <= 50 and size <= 5
else 'FAIL'])
B12 = pd.DataFrame(rows, columns=['model', 'acc clean-train', 'acc
noisy-train', 'drop', 'latency ms (Colab)', 'latency ms (phone est.)',
'size MB', '50ms & 5MB'])
display(B12)
```


```
{"summary":"{\n  \"name\": \"B12\",\n  \"rows\": 4,\n  \"fields\": [\n
{\n      \"column\": \"model\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 4,\n
\"samples\": [\n          \"k-NN k=1 + PCA\",\n
\"RandomForest + PCA\",\n          \"LogReg + PCA\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"acc clean-train\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.04927646497061249,\n        \"min\": 0.1865,\n        \"max\":
0.3023,\n        \"num_unique_values\": 4,\n        \"samples\": [\n
0.3023,\n          0.2737,\n          0.1865\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"acc noisy-train\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.04356300992049715,\n        \"min\": 0.1497,\n        \"max\":
0.2427,\n        \"num_unique_values\": 4,\n        \"samples\": [\n
0.2368,\n          0.2287,\n          0.1497\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"drop\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.02105348348057078,\n
\"min\": 0.0146,\n        \"max\": 0.0655,\n
\"num_unique_values\": 4,\n        \"samples\": [\n          0.0655,\n
0.045,\n          0.0368\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"latency ms (Colab)\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 3.2883323341779187,\n
\"min\": 1.6977,\n        \"max\": 9.7376,\n
\"num_unique_values\": 4,\n        \"samples\": [\n          6.1332,\n
9.7376,\n          1.6977\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    },\n    {\n
\"column\": \"latency ms (phone est.)\",\n      \"properties\": {\n
```

\"dtype\": \"number\",\n \"std\": 26.30644713994398,\n
\"min\": 13.5819,\n \"max\": 77.9006,\n
\"num_unique_values\": 4,\n \"samples\": [\n 49.0654,\
n77.9006,\n13.5819\n],\n
\"semantic_type\": \"\",\n\"description\": \"\"\n}\
n },\n {\n \"column\": \"size MB\",\n \"properties\":
{\n \"dtype\": \"number\",\n \"std\":
54.10565973484197,\n\"min\": 8.9494,\n\"max\": 122.4575,\n\"num_unique_values\":
3,\n\"samples\": [\n 8.9494,\n17.3276,\n122.4575\n],\n
\"semantic_type\": \"\",\n\"description\": \"\"\n}\ n},\n{\n\"column\": \"50ms & 5MB\",\n
\"properties\": {\n\"dtype\": \"category\",\n
\"num_unique_values\": 1,\n\"samples\": [\n\"FAIL\"\ n],\n\"semantic_type\": \"\",\n
\"description\": \"\"\n}\n}\n]\ n}","type":"dataframe","variable_name":"B12"}

B. Examination: check whether the Task 9/11 winner passes (k-NN stores every training image → size grows with data; RF size grows with trees×depth). Which model drops most with noise? (k=1 memorises flipped labels; k=15 votes them away; LR's regularised linear boundary averages over noise.)

C. Adaptation options – (1) PCA dimension / regularisation, (2) label smoothing in the from-scratch softmax, (3) model compression.

```
# Option 1: PCA dimension x regularisation strength for LogReg (trained on NOISY labels, tested on clean)
rows = []
                                                                for k in [20, 50, 100, K_PCA]:
                                                                    for C in [0.01, 0.1, 1.0]:
m = make_pipeline(PCA(k, random_state=SEED), StandardScaler(), LogisticRegression(C=C,
max_iter=300)).fit(X38_tr, y38_noisy)
rows.append([k, C, f4(accuracy_score(y38_te, m.predict(X38_te))), f4(latency_ms(m, X38_te, 50) *
PHONE_SLOWDOWN), f4(len(pickle.dumps(m)) / 2**20),
(k + 1) * K38 + k * X38.shape[1]])
opt1 = pd.DataFrame(rows, columns=['PCA k', 'C', 'clean test acc', 'phone ms est.', 'size MB', 'params']);
display(opt1)
```

{"summary":"{\n \"name\": \"opt1\",\n \"rows\": 12,\n \"fields\": [\n{\n\"column\": \"PCA
k\",\n\"properties\": {\n
\"dtype\": \"number\",\n\"std\": 229,\n \"min\": 20,\n
\"max\": 560,\n\"num_unique_values\": 4,\n \"samples\":
[\n50,\n560,\n20\n
\"semantic_type\": \"\",\n\"description\": \"\"\n}\
n},\n{\n\"column\": \"C\",\n\"properties\": {\n
\"dtype\": \"number\",\n \"std\": 0.46686575839703254,\n
\"min\": 0.01,\n \"max\": 1.0,\n \"num_unique_values\":

```
3,\n        \"samples\": [\n          0.01,\n          0.1,\n
1.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"clean test acc\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.055864730430394145,\n        \"min\":
0.1497,\n        \"max\": 0.3094,\n \"num_unique_values\":       12,\
n        \"samples\": [\n          0.1637,\n          0.1854,\n
0.2526\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"phone ms est.\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n    \"std\": 2.368926690028054,\n            \"min\":
7.1466,\n        \"max\": 14.0108,\n        \"num_unique_values\":
12,\n        \"samples\": [\n12.6999,\n     12.9833,\n
12.9118\n        ],\n\"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"size MB\",\n      \"properties\": {\n        \"dtype\": \"number\",\
n        \"std\": 3.6617838264728553,\n        \"min\": 0.3368,\n
\"max\": 8.9494,\n        \"num_unique_values\": 4,\n
\"samples\": [\n0.8153,\n 8.9494,\n
0.3368\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"params\",\n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 949123,\n        \"min\": 82718,\n        \"max\": 2315078,\n
\"num_unique_values\": 4,\n        \"samples\": [\n          206738,\n
2315078,\n          82718\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    }\n ]\
n}","type":"dataframe","variable_name":"opt1"}
```


Option 2 — Label Smoothing

Smoothed target

$$\tilde{y}_k = (1-\varepsilon)\,1[k=y] + \frac{\varepsilon}{K}$$

Trained with the from-scratch softmax regression (Task 8) on the noisy labels, tested on clean labels.

```
# Option 2: label smoothing (from-scratch softmax regression, Task 8
code): target = (1-ε)·onehot + ε/K
pz = PCA(100 if not TEST_MODE else 30, random_state=SEED).fit(X38_tr);
sz = StandardScaler().fit(pz.transform(X38_tr))
G_tr, G_te = add_bias(sz.transform(pz.transform(X38_tr))),
add_bias(sz.transform(pz.transform(X38_te)))
rows = []
for eps in [0.0, 0.1, 0.2, 0.3]:
    Yt = (1 - eps) * np.eye(K38)[y38_noisy] + eps / K38
    W, hist, _, _ = train_softmax(G_tr, Yt, 0.05, 32, 10 if TEST_MODE
else 40)
```

```
    Pq = softmax_rows(G_te @ W)
    rows.append([eps, f4(accuracy_score(y38_te, Pq.argmax(1))),
f4(np.mean(-np.log2(np.clip(Pq[np.arange(len(y38_te)), y38_te], 1e-12,
1)))), f4(Pq.max(1).mean())])
opt2 = pd.DataFrame(rows, columns=['ε (smoothing)', 'clean test acc',
'test CE (bits)', 'mean max-prob (confidence)']); display(opt2)
```


```
{"summary":"{\n  \"name\": \"opt2\",\n  \"rows\": 4,\n  \"fields\": [\
n    {\n      \"column\": \"\\u03b5 (smoothing)\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.12909944487358055,\n        \"min\": 0.0,\n        \"max\": 0.3,\n
\"num_unique_values\": 4,\n        \"samples\": [\n          0.1,\n
0.3,\n          0.0\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"clean test acc\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.0033619190948028454,\n        \"min\":
0.2602,\n        \"max\": 0.2673,\n        \"num_unique_values\": 3,\n
\"samples\": [\n0.2602,\n 0.2643,\n
0.2673\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"test CE (bits)\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 0.027687843301106862,\n        \"min\":
4.0479,\n        \"max\": 4.1008,\n        \"num_unique_values\": 4,\n
\"samples\": [\n4.0479,\n 4.0916,\n
4.1008\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"mean max-prob (confidence)\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 0.05696059310318553,\n
\"min\": 0.1717,\n        \"max\": 0.3038,\n
\"num_unique_values\": 4,\n        \"samples\": [\n          0.2534,\n
0.1717,\n          0.3038\n        ],\n        \"semantic_type\":
\"\",\n        \"description\": \"\"\n      }\n    }\n ]\
n}","type":"dataframe","variable_name":"opt2"}
```


Option 3 — Compress / Simplify the Model

Model size

$$\text{size} = \text{¿ params} \times \text{bytes per param (float32=4 ,float16=2)}$$

Fewer/shallower trees for the random forest; half-precision weights for the PCA + logistic-regression model.

```
# Option 3: compress / simplify the model
rows = []
for nm, m in {'RF 100 trees, full depth': RandomForestClassifier(100,
random_state=SEED, n_jobs=1),
              'RF 30 trees, depth 12': RandomForestClassifier(30,
max_depth=12, random_state=SEED, n_jobs=1),
```

```
            'RF 10 trees, depth 8': RandomForestClassifier(10,
max_depth=8, random_state=SEED, n_jobs=1)}.items():
    pipe = make_pipeline(PCA(K_PCA, random_state=SEED), m).fit(X38_tr,
y38_noisy)
    rows.append([nm, f4(accuracy_score(y38_te, pipe.predict(X38_te))),
f4(latency_ms(pipe, X38_te, 50) * PHONE_SLOWDOWN),
f4(len(pickle.dumps(pipe)) / 2**20)])
# float16 weights for the linear model (PCA basis + LR weights)
lr16 = make_pipeline(PCA(50, random_state=SEED), StandardScaler(),
LogisticRegression(C=0.1, max_iter=300)).fit(X38_tr, y38_noisy)
pc, sc_, lr_ = lr16.steps[0][1], lr16.steps[1][1], lr16.steps[2][1]
Wf = [pc.components_.astype(np.float16), pc.mean_.astype(np.float16),
lr_.coef_.astype(np.float16), lr_.intercept_.astype(np.float16)]
z16 = ((X38_te - Wf[1].astype(np.float32)) @
Wf[0].astype(np.float32).T - sc_.mean_) / sc_.scale_
acc16 = accuracy_score(y38_te, (z16 @ Wf[2].astype(np.float32).T +
Wf[3]).argmax(1))
rows.append(['LogReg PCA-50 float32', f4(accuracy_score(y38_te,
lr16.predict(X38_te))), f4(latency_ms(lr16, X38_te, 50) *
PHONE_SLOWDOWN), f4(sum(w.size for w in Wf) * 4 / 2**20)])
rows.append(['LogReg PCA-50 float16', f4(acc16), np.nan,
f4(sum(w.nbytes for w in Wf) / 2**20)])
opt3 = pd.DataFrame(rows, columns=['model', 'clean test acc (noisy
train)', 'phone ms est.', 'size MB']); display(opt3)
```


```
{"summary":"{\n  \"name\": \"opt3\",\n  \"rows\": 5,\n  \"fields\": [\
n    {\n      \"column\": \"model\",\n      \"properties\": {\n
\"dtype\": \"string\",\n        \"num_unique_values\": 5,\n
\"samples\": [\n          \"RF 30 trees, depth 12\",\n
\"LogReg PCA-50 float16\",\n          \"RF 10 trees, depth 8\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"clean test acc (noisy train)\",\
n      \"properties\": {\n        \"dtype\": \"number\",\n
\"std\": 0.07547055717297972,\n        \"min\": 0.1368,\n
\"max\": 0.3058,\n        \"num_unique_values\": 4,\n
\"samples\": [\n0.1789,\n 0.3058,\n
0.2287\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"phone ms est.\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 30.019000076770823,\n        \"min\":
11.3724,\n        \"max\": 79.4891,\n        \"num_unique_values\":
4,\n        \"samples\": [\n          33.5809,\n          11.3724,\n
79.4891\n        ],\n\"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"size MB\",\n      \"properties\": {\n        \"dtype\": \"number\",\
n        \"std\": 52.40064709860938,\n        \"min\": 0.4021,\n
\"max\": 122.4575,\n        \"num_unique_values\": 5,\n
\"samples\": [\n          13.2278,\n          0.4021,\n          9.31\
n        ],\n        \"semantic_type\": \"\",\n
```

\"description\": \"\"\n   }\n   }\n]\
n}","type":"dataframe","variable_name":"opt3"}

C. Points to analyse: for each option state the effect on capacity (fewer PCs / smaller C / shallower trees ↓; smoothing doesn't change capacity but stops it fitting noisy one-hot targets with full confidence), error (bias ↑ vs variance ↓ under noise) and cost (MB, ms).

```
checks = pd.concat([B12[['model', 'acc noisy-train', 'latency ms
(phone est.)', 'size MB']].rename(columns={'acc noisy-train': 'clean
test acc (noisy train)', 'latency ms (phone est.)': 'phone ms est.'}),
                    opt3.rename(columns={'clean test acc (noisy
train)': 'clean test acc (noisy train)'})], ignore_index=True)
checks['meets 50ms & 5MB'] = (checks['phone ms est.'].fillna(0) <= 50)
& (checks['size MB'] <= 5)
display(checks.sort_values(['meets 50ms & 5MB', 'clean test acc (noisy
train)'], ascending=False))
```

{"summary":"{\n \"name\": \"display(checks\",\n \"rows\": 9,\n
\"fields\": [\n   {\n   \"column\": \"model\",\n
\"properties\": {\n   \"dtype\": \"string\",\n
\"num_unique_values\": 9,\n   \"samples\": [\n   \"LogReg
+ PCA\",\n   \"LogReg PCA-50 float16\",\n   \"RF 100
trees, full depth\"\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   },\n   {\n   \"column\":
\"clean test acc (noisy train)\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 0.060309659259524924,\n
\"min\": 0.1368,\n   \"max\": 0.3058,\n
\"num_unique_values\": 7,\n   \"samples\": [\n   0.3058,\n
0.2427,\n   0.1497\n   ],\n   \"semantic_type\":
\"\",\n   \"description\": \"\"\n   }\n   },\n   {\n
\"column\": \"phone ms est.\",\n   \"properties\": {\n
\"dtype\": \"number\",\n   \"std\": 26.685578376367445,\n
\"min\": 11.3724,\n   \"max\": 79.4891,\n
\"num_unique_values\": 8,\n   \"samples\": [\n   46.2457,\
n   33.5809,\n   11.3724\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"size MB\",\n   \"properties\":
{\n   \"dtype\": \"number\",\n   \"std\":
50.125371976818734,\n   \"min\": 0.4021,\n   \"max\":
122.4575,\n\"num_unique_values\": 7,\n\"samples\":   [\n
0.8043,\n   0.4021,\n   8.9494\n   ],\n
\"semantic_type\": \"\",\n   \"description\": \"\"\n   }\
n   },\n   {\n   \"column\": \"meets 50ms & 5MB\",\n
\"properties\": {\n   \"dtype\": \"boolean\",\n
\"num_unique_values\": 2,\n   \"samples\": [\n   false,\n
true\n   ],\n   \"semantic_type\": \"\",\n
\"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe"}

D. Evidence-based conclusion — RETAIN / MODIFY / REDESIGN

Write your decision here and trace each element to:

1. Mathematics – entropy ceiling $\log_2 38$ (Part A), bias–variance under noise (Task 7 + Part B drop), conditioning/whitening (Task 4, 8).
2. Experiments – accuracies in Part B and options in Part C.
3. Measurements – phone-latency and MB columns in the table above (state the PHONE_SLOWDOWNassumption).
4. Trade-offs – which option gives the best accuracy *within* 50 ms / 5 MB.